

EECS 189: Introduction to Machine Learning

Comprehensive Midterm Study Notes

Lectures 1–16, Discussions 1–7, Homeworks 1–2

UC Berkeley – Spring 2026

Contents

1	Introduction to Machine Learning	4
1.1	What is Machine Learning?	4
1.2	Types of Machine Learning	4
1.3	Key Concepts	4
1.4	Simple Models	5
1.5	Error Types and Evaluation	5
1.6	Parametric vs. Non-Parametric Models	5
2	Data Preprocessing and Feature Engineering	5
2.1	Feature Engineering	5
2.2	The Bias Trick	6
2.3	Basis Expansions	6
2.4	Data Splitting	6
3	Probability and Statistics Review	7
3.1	Fundamental Rules	7
3.2	Independence	8
3.3	Expectation, Variance, and Covariance	8
3.4	Important Distributions	9
4	Linear Algebra and Matrix Calculus	9
4.1	Matrix Calculus Identities	9
4.2	Hessian Matrix	10
4.3	Positive Semi-Definite and Positive Definite Matrices	11
4.4	SVD and Pseudoinverse	11
4.5	Eigendecomposition and Matrix Powers	11
4.6	Rotation Matrices	12
5	Clustering	12
5.1	K-Means Clustering	12
5.2	K-Means: Practical Considerations	13
5.3	Robust K-Means	13

6	Gaussian Models	13
6.1	Univariate Gaussian	13
6.2	Multivariate Gaussian	14
6.3	MLE for Multivariate Gaussian	14
6.4	Mixture of Gaussians (GMM)	15
7	Maximum Likelihood Estimation	16
7.1	The MLE Framework	16
7.2	MLE Examples	16
7.3	MAP Estimation	17
8	Linear Regression	18
8.1	Problem Setup	18
8.2	Ordinary Least Squares (OLS)	18
8.3	Geometric Interpretation	19
8.4	When $X^\top X$ is Not Invertible	19
8.5	Ridge Regression (L2 Regularization)	19
8.6	Lasso Regression (L1 Regularization)	20
8.7	Elastic Net	20
8.8	Weighted Least Squares	20
8.9	Regression Metrics	21
9	Bias-Variance Tradeoff	21
9.1	Setup	21
9.2	Full Proof of the Decomposition	22
9.3	Interpretation	22
9.4	The Tradeoff	23
9.5	K-Fold Cross-Validation	24
10	Classification	24
10.1	Three Approaches to Classification	24
10.2	Gaussian Discriminant Analysis (GDA)	25
10.2.1	MLE for GDA Parameters	25
10.2.2	Deriving the Decision Boundary	26
10.3	LDA vs QDA: The Four Cases	28
10.4	Naive Bayes	30
10.5	Logistic Regression	30
10.5.1	The Sigmoid Function	30
10.5.2	Logistic Regression Model	31
10.5.3	MLE for Logistic Regression (Full Derivation)	32
10.5.4	Multi-Class: Softmax Regression	34
10.6	Evaluating Classifiers	34
10.6.1	Confusion Matrix and Metrics	34
10.6.2	ROC Curve	36

11 Optimization	38
11.1 Batch Gradient Descent	38
11.2 Convergence Analysis (Detailed)	39
11.3 Stochastic Gradient Descent (SGD)	41
11.4 Momentum (Detailed Derivation)	42
11.5 Adaptive Learning Rate Methods	44
11.6 Learning Rate Schedules	46
12 Loss Functions for Classification	46
12.1 The Three Main Classification Losses	46
12.2 Comparing Loss Functions Graphically	47
12.3 Multi-Class Cross-Entropy Loss	48
13 Information Theory	49
13.1 Entropy	49
13.2 KL Divergence	49
13.3 Cross-Entropy	50
14 Key Tricks, Patterns, and Problem-Solving Techniques	50
14.1 Sigmoid Identities	50
14.2 Gradient Computation Patterns	50
14.3 Normal Equations Pattern	51
14.4 Residual Orthogonality	51
14.5 Chain Rule for Deep/Recursive Systems	51
14.6 PSD/PD Proof Technique	51
14.7 Overparameterized Systems and Min-Norm Solutions	52
14.8 Likelihood Ratio and Capture-Recapture MLE	52
14.9 Tail Probability Formula for Expectation	52
14.10 Robust Methods	52
14.11 p -Norms	52
14.12 Model Relationships Summary	53
14.13 Regularization Summary	53

1 Introduction to Machine Learning

1.1 What is Machine Learning?

Machine learning is the study of algorithms that improve their performance at some task through experience (data). Rather than explicitly programming rules, we let the algorithm *learn* patterns from data.

Definition 1.1 (The ML Pipeline). The machine learning pipeline consists of four stages:

1. **Learning Problem:** Define what you want to predict and what data you have.
2. **Model Design:** Choose a model family (hypothesis space) and features.
3. **Optimization:** Find the best parameters within the model family.
4. **Predict & Evaluate:** Use the trained model on new data and assess performance.

1.2 Types of Machine Learning

Definition 1.2 (Supervised Learning). Given a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ of input-output pairs, learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that maps inputs to outputs.

- **Classification:** y is discrete/categorical (e.g., spam vs. not spam).
- **Regression:** y is continuous/real-valued (e.g., house price prediction).

Definition 1.3 (Unsupervised Learning). Given a dataset $D = \{\mathbf{x}_i\}_{i=1}^N$ with no labels, discover hidden structure. Includes:

- **Clustering:** Group similar data points (K-means, GMM).
- **Dimensionality Reduction:** Find lower-dimensional representations.
- **Density Estimation:** Estimate the probability distribution $p(\mathbf{x})$.
- **Generative Modeling:** Learn to generate new data samples.

Definition 1.4 (Reinforcement Learning). An agent interacts with an environment, taking actions and receiving rewards. The goal is to learn a policy that maximizes cumulative reward.

1.3 Key Concepts

Definition 1.5 (Hypothesis Space). The set of all possible models within a chosen model family. For example, the hypothesis space for linear classifiers in 2D is the set of all lines $w_1x_1 + w_2x_2 + b = 0$.

Definition 1.6 (Inductive Bias). Assumptions built into the model design that allow it to generalize beyond the training data. Every model has inductive biases—the choice of model family itself is a bias.

Key Idea 1.1 (No Free Lunch Theorem). No single model is universally best across all possible problems. The best model depends on the specific problem structure. This is why understanding your data and problem is essential.

Definition 1.7 (Generalization). The ability of a trained model to perform well on new, unseen data drawn from the same distribution as the training data. This is the ultimate goal of ML—we don't just want to memorize training data.

1.4 Simple Models

Decision Trees: Partition the feature space using axis-aligned splits. A *decision stump* (depth 1) applies a single threshold on one feature: $f(x) = 1$ if $x \geq \text{threshold}$. Deeper trees apply sequences of splits, creating rectangular decision regions.

Linear Classifiers: Predict using $f(\mathbf{x}) = 1$ if $\mathbf{w}^\top \mathbf{x} + b \geq 0$, else 0. The decision boundary is the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$, and $\mathbf{w}$ is the normal vector to this hyperplane.

Nearest Neighbor: Assign the label of the closest training point: $g_D(\mathbf{x}) = y_{i(\mathbf{x})}$ where $i(\mathbf{x}) = \operatorname{argmin}_n \|\mathbf{x}_n - \mathbf{x}\|$. Decision regions form a *Voronoi diagram*. This is a *non-parametric* method—the number of effective parameters grows with the data.

1.5 Error Types and Evaluation

Definition 1.8 (Confusion Matrix). For binary classification:

	Predicted −	Predicted +
Actual −	True Negative (TN)	False Positive (FP)
Actual +	False Negative (FN)	True Positive (TP)

- **Type I Error** (False Positive): Predict positive when actually negative.
- **Type II Error** (False Negative): Predict negative when actually positive.

1.6 Parametric vs. Non-Parametric Models

Parametric models have a fixed number of parameters regardless of data size (e.g., linear regression, logistic regression). **Non-parametric models** have a number of parameters that grows with the data (e.g., KNN, decision trees).

2 Data Preprocessing and Feature Engineering

2.1 Feature Engineering

Feature engineering is the process of transforming raw data into features that better represent the underlying problem, making the learning algorithm more effective.

Definition 2.1 (Feature Standardization (Z-score Normalization)). Transform each feature to have zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}$$

Critical: Compute μ and σ from the *training data only*. Apply these same statistics when transforming test/validation data. Never use test data statistics—this would be data leakage.

Definition 2.2 (One-Hot Encoding). Convert a categorical variable with K categories into K binary indicator columns. For example, $\text{color} \in \{\text{red}, \text{blue}, \text{green}\}$ becomes three binary features: $[\mathbf{1}_{\text{red}}, \mathbf{1}_{\text{blue}}, \mathbf{1}_{\text{green}}]$.

Bag-of-Words: Represent a text document as a vector of word counts (or frequencies). Common to remove stop words (“the”, “is”, etc.).

2.2 The Bias Trick

Instead of writing $\hat{y} = \mathbf{w}^\top \mathbf{x} + w_0$ with a separate intercept term, we can append a 1 to the feature vector: $\mathbf{x}' = [\mathbf{x}; 1]$, and then $\hat{y} = \mathbf{w}'^\top \mathbf{x}'$ absorbs the bias into the weight vector. This simplifies notation and derivations.

2.3 Basis Expansions

Definition 2.3 (Basis Expansion). A mapping $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$ that transforms the original features into a new (often higher-dimensional) feature space:

$$\hat{y} = \mathbf{w}^\top \Phi(\mathbf{x})$$

The model is still *linear in the parameters* $\mathbf{w}$, even though $\Phi(\mathbf{x})$ may be a nonlinear function of $\mathbf{x}$.

Common basis expansions:

- **Polynomial:** $\Phi(x) = [1, x, x^2, \dots, x^p]$ for degree p .
- **Polynomial (2D, quadratic):** $\Phi(x_1, x_2) = [1, x_1, x_2, x_1x_2, x_1^2, x_2^2]$.
- **RBF (Radial Basis Functions):** $\phi_j(\mathbf{x}) = \exp(-\|\mathbf{x} - \boldsymbol{\mu}_j\|^2/r)$ centered at various points $\boldsymbol{\mu}_j$.
- **Fourier / sinusoidal** features for periodic data.

2.4 Data Splitting

Key Idea 2.1 (Train / Validation / Test Split). • **Training set (~60–80%):** Used to fit model parameters (weights $\mathbf{w}$).

- **Validation set (~10–20%):** Used to select hyperparameters (λ , degree, K , etc.). Can evaluate multiple times.
- **Test set (~10–20%):** Used *once* for final evaluation. Never use to make modeling decisions.

Using the test set for model selection leads to optimistic estimates of generalization performance.

3 Probability and Statistics Review

3.1 Fundamental Rules

Definition 3.1 (Joint, Marginal, and Conditional Probability). For random variables X and Y :

- **Joint probability:** $p(X = x_i, Y = y_j)$ – probability of both events.
- **Marginal (Sum Rule):** $p(X = x_i) = \sum_j p(X = x_i, Y = y_j)$ – “sum out” the variable you don’t need.
- **Conditional:** $p(Y = y_j | X = x_i) = \frac{p(X=x_i, Y=y_j)}{p(X=x_i)}$

Definition 3.2 (Product Rule (Chain Rule)).

$$p(X, Y) = p(Y | X) p(X) = p(X | Y) p(Y)$$

For three variables: $p(X, Y, Z) = p(Z | X, Y) p(X | Y) p(Y)$.

This generalizes to any number of variables.

Theorem 3.1 (Bayes’ Theorem).

$$p(A | B) = \frac{p(B | A) p(A)}{p(B)} = \frac{p(B | A) p(A)}{\sum_a p(B | A = a) p(A = a)}$$

- $p(A)$: **prior** – our belief about A before seeing data.
- $p(B | A)$: **likelihood** – how likely the observed data is under hypothesis A .
- $p(A | B)$: **posterior** – updated belief after seeing data B .
- $p(B)$: **evidence** (normalizing constant) – ensures posterior sums to 1.

Example 3.1 (Base Rate Fallacy (Wake Word Detector)). Suppose a wake word detector has:

- Prior: $P(\text{wake}) = 0.0001$ (wake words are rare)
- True positive rate: $P(\text{detect} | \text{wake}) = 0.99$
- False positive rate: $P(\text{detect} | \text{no wake}) = 0.001$

By Bayes’ theorem:

$$P(\text{wake} | \text{detect}) = \frac{0.99 \times 0.0001}{0.99 \times 0.0001 + 0.001 \times 0.9999} = \frac{0.000099}{0.001098} \approx 0.09$$

Even with 99% recall and 0.1% FPR, the posterior is only $\sim 9\%$ because the prior is so low. This is the *base rate fallacy*.

3.2 Independence

Definition 3.3 (Independence and Conditional Independence). X and Y are **independent** ($X \perp Y$) if and only if:

$$p(X, Y) = p(X) \cdot p(Y) \quad \Leftrightarrow \quad p(Y | X) = p(Y)$$

Knowing X provides no information about Y .

X and Y are **conditionally independent** given Z ($X \perp Y | Z$) if:

$$p(X, Y | Z) = p(X | Z) \cdot p(Y | Z)$$

This is the key assumption behind **Naive Bayes** classifiers.

Definition 3.4 (IID (Independent and Identically Distributed)). Data points $x_1, \dots, x_N$ are IID if they are mutually independent and all drawn from the same distribution:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n)$$

This is a standard assumption in ML. It simplifies likelihoods into products.

3.3 Expectation, Variance, and Covariance

Definition 3.5 (Expectation). The expected value (mean) of a random variable:

$$\mathbb{E}[X] = \sum_x x \cdot p(x) \quad (\text{discrete}) \quad \mathbb{E}[X] = \int x \cdot p(x) dx \quad (\text{continuous})$$

Linearity of expectation (always holds, even without independence):

$$\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c$$

For independent RVs: $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

Definition 3.6 (Variance). Measures spread around the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Properties: $\text{Var}(aX + b) = a^2 \text{Var}(X)$. For *independent* RVs: $\text{Var}(\sum a_i X_i) = \sum a_i^2 \text{Var}(X_i)$.

Definition 3.7 (Covariance and Correlation).

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

Note: $\text{Cov}(X, X) = \text{Var}(X)$.

Correlation normalizes covariance to $[-1, 1]$:

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \cdot \text{Var}(Y)}}$$

$\text{Cov}(X, Y) = 0$ means X and Y are uncorrelated, but this does NOT imply independence in general (exception: multivariate Gaussians).

3.4 Important Distributions

Definition 3.8 (Key Distributions).

• **Bernoulli(p)**: $P(X = 1) = p$, $P(X = 0) = 1 - p$.
Mean = p , Variance = $p(1 - p)$.

• **Binomial(n, p)**: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$. Sum of n iid Bernoulli trials.

• **Poisson(λ)**: $P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$. Models rare event counts. Mean = Variance = λ .

• **Gaussian(μ, σ^2)**: $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

• **Beta(α, β)**: $p(y) = \frac{1}{B(\alpha, \beta)} y^{\alpha-1} (1-y)^{\beta-1}$, $y \in [0, 1]$. Conjugate prior for Bernoulli.

• **Laplace(μ, b)**: $p(z) = \frac{1}{2b} \exp(-|z - \mu|/b)$. Sharper peak at μ than Gaussian. Related to L1 regularization.

Example 3.2 (Useful Probability Result: Minimum of IID Random Variables). If $X_1, \dots, X_n$ are iid with CDF F , then for $M = \min(X_1, \dots, X_n)$:

$$P(M > t) = \prod_{i=1}^n P(X_i > t) = (1 - F(t))^n$$

For n iid Uniform $[0, 1]$: $P(M > t) = (1 - t)^n$, so $f_M(t) = n(1 - t)^{n-1}$.

Tail probability formula for expectation: $\mathbb{E}[X] = \int_0^\infty P(X > t) dt$ for non-negative RVs.

4 Linear Algebra and Matrix Calculus

4.1 Matrix Calculus Identities

These identities are used repeatedly in ML derivations. You should know them cold.

Theorem 4.1 (Essential Gradient Identities). Let $\mathbf{w}, \mathbf{a} \in \mathbb{R}^d$, $A \in \mathbb{R}^{d \times d}$, $X \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{w}}(\mathbf{a}^\top \mathbf{w}) = \mathbf{a} \quad (1)$$

$$\nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{w}) = 2\mathbf{w} \quad (2)$$

$$\nabla_{\mathbf{w}}(\mathbf{w}^\top A \mathbf{w}) = (A + A^\top) \mathbf{w} \quad (\text{if } A \text{ symmetric: } = 2A \mathbf{w}) \quad (3)$$

$$\nabla_{\mathbf{w}} \|X \mathbf{w} - \mathbf{y}\|^2 = 2X^\top (X \mathbf{w} - \mathbf{y}) \quad (\text{most important!}) \quad (4)$$

The last one is derived by expanding $\|X \mathbf{w} - \mathbf{y}\|^2 = \mathbf{w}^\top X^\top X \mathbf{w} - 2\mathbf{y}^\top X \mathbf{w} + \mathbf{y}^\top \mathbf{y}$ and differentiating.

Theorem 4.2 (Matrix Calculus for Log-Determinant and Trace). For invertible matrix A and arbitrary matrix B :

$$\nabla_A \log \det(A) = A^{-1} \quad (5)$$

$$\nabla_A \text{Tr}(A^{-1} B) = -A^{-1} B A^{-1} \quad (6)$$

These are used in deriving the MLE for the covariance matrix of a multivariate Gaussian.

4.2 Hessian Matrix

Definition 4.1 (Hessian). The Hessian of $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the $d \times d$ matrix of second partial derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial w_i \partial w_j}$$

Key Hessians:

- $f(\mathbf{w}) = \|\mathbf{w}\|^2$: $H = 2I$
- $f(\mathbf{w}) = \mathbf{w}^\top A \mathbf{w}$: $H = A + A^\top$
- $f(\mathbf{w}) = \|X \mathbf{w} - \mathbf{y}\|^2$: $H = 2X^\top X$

Definition 4.2 (Critical Point Classification via Hessian). At a critical point (where gradient = 0):

- All eigenvalues of $H > 0$ (positive definite) $\Rightarrow$ **local minimum**.
- All eigenvalues of $H < 0$ (negative definite) $\Rightarrow$ **local maximum**.
- Mixed positive and negative eigenvalues $\Rightarrow$ **saddle point**.

For 2D: $\det(H) > 0$ and $H_{11} > 0 \Rightarrow \text{min}$; $\det(H) > 0$ and $H_{11} < 0 \Rightarrow \text{max}$; $\det(H) < 0 \Rightarrow \text{saddle}$.

4.3 Positive Semi-Definite and Positive Definite Matrices

Definition 4.3 (PSD and PD). A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is:

- **Positive Semi-Definite (PSD)** if $\mathbf{x}^\top A \mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$. Equivalently, all eigenvalues ≥ 0 .
- **Positive Definite (PD)** if $\mathbf{x}^\top A \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$. Equivalently, all eigenvalues > 0 .

Theorem 4.3 (PSD/PD Characterizations). • A is PSD $\Leftrightarrow$ all eigenvalues $\geq 0 \Leftrightarrow A = B^\top B$ for some matrix B .

- A is PD $\Leftrightarrow$ all eigenvalues $> 0 \Leftrightarrow A = B^\top B$ for some invertible matrix B .
- $A^\top A$ is **always PSD** for any matrix A (since $\mathbf{x}^\top A^\top A \mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$).
- $A^\top A$ is PD if and only if A has full column rank.

Proof (spectral decomposition): If $A = Q\Lambda Q^\top$ (eigendecomposition), then $\mathbf{x}^\top A \mathbf{x} = \sum_i \lambda_i (q_i^\top \mathbf{x})^2$. This is ≥ 0 for all $\mathbf{x}$ iff all $\lambda_i \geq 0$.

4.4 SVD and Pseudoinverse

Definition 4.4 (Singular Value Decomposition (SVD)). Any matrix $X \in \mathbb{R}^{n \times d}$ can be decomposed as $X = U\Sigma V^\top$ where U is $n \times n$ orthogonal, Σ is $n \times d$ diagonal (with singular values σ_i), and V is $d \times d$ orthogonal.

The singular values are $\sigma_i = \sqrt{\lambda_i(X^\top X)}$.

Definition 4.5 (Pseudoinverse). The Moore-Penrose pseudoinverse $X^+ = V\Sigma^+U^\top$.

Overdetermined system ($n > d$, more equations than unknowns):

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y} \quad (\text{least squares solution})$$

Underdetermined system ($d > n$, more unknowns than equations):

$$\mathbf{w}^* = X^\top (X X^\top)^{-1} \mathbf{y} \quad (\text{minimum norm solution})$$

This is the solution with the smallest $\|\mathbf{w}\|_2$, and it's what gradient descent from $\mathbf{w}_0 = \mathbf{0}$ converges to.

Theorem 4.4 (Maximum of $\|A\mathbf{x}\|$). $\max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\| = \sigma_{\max}$ (the largest singular value of A). The maximizer is the corresponding right singular vector (eigenvector of $A^\top A$ for the largest eigenvalue).

4.5 Eigendecomposition and Matrix Powers

For a diagonalizable matrix $M = PDP^{-1}$:

$$M^n = PD^nP^{-1}$$

Three convergence cases for M^n as $n \rightarrow \infty$:

- All $|\lambda_i| < 1$: $M^n \rightarrow \mathbf{0}$ (converges to zero matrix).

- Some $|\lambda_i| > 1$: M^n diverges.
- $|\lambda_i| = 1$ but $\lambda_i \neq 1$: M^n oscillates (no limit).

4.6 Rotation Matrices

The 2D rotation matrix for counterclockwise rotation by angle θ :

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Key property: $R(\phi)R(\theta) = R(\theta + \phi)$. This composition gives the angle addition formulas for sine and cosine.

5 Clustering

5.1 K-Means Clustering

Definition 5.1 (K-Means Objective). Partition n data points into K clusters $C_1, \dots, C_K$ with centroids $\mathbf{c}_1, \dots, \mathbf{c}_K$ to minimize:

$$J = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mathbf{c}_k\|^2$$

This measures the total within-cluster sum of squared distances.

Definition 5.2 (Lloyd's Algorithm (K-Means)). 1. **Initialize:** Choose K initial centroids (e.g., randomly from data points).

2. **Repeat until convergence:**

(a) **Assignment step:** Assign each point to its nearest centroid:

$$z_i = \operatorname{argmin}_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \mathbf{c}_k\|^2$$

(b) **Update step:** Recompute each centroid as the mean of its assigned points:

$$\mathbf{c}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$$

Theorem 5.1 (K-Means Convergence). Lloyd’s algorithm is guaranteed to converge in a finite number of iterations.

Proof sketch:

1. The assignment step cannot increase J (each point moves to its nearest centroid).
2. The update step cannot increase J (the mean minimizes sum of squared distances).
3. J decreases (or stays the same) at each step.
4. There are finitely many possible cluster assignments.
5. Therefore, the algorithm must terminate.

However, convergence is to a **local minimum**, not necessarily the global minimum.

Centroid optimality derivation: Given fixed assignments, we want $\operatorname{argmin}_{\mathbf{c}_k} \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mathbf{c}_k\|^2$. Taking the derivative and setting to zero:

$$\frac{\partial}{\partial \mathbf{c}_k} \sum_{\mathbf{x} \in C_k} (\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{x} - \mathbf{c}_k) = -2 \sum_{\mathbf{x} \in C_k} (\mathbf{x} - \mathbf{c}_k) = 0 \implies \mathbf{c}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$$

5.2 K-Means: Practical Considerations

- **Choosing K :** Use the *elbow method*—plot J vs. K and look for a “knee” where adding more clusters gives diminishing returns.
- **Initialization sensitivity:** Run multiple times with different random initializations and keep the best result.
- **Weaknesses:**
 - Assumes spherical, equally-sized clusters.
 - Hard assignments—each point belongs to exactly one cluster.
 - Sensitive to outliers (since it minimizes squared distances).
 - Sensitive to initialization—different starting points can give very different results.

5.3 Robust K-Means

Standard K-means is sensitive to outliers because the squared loss $\|x - c\|^2$ grows quadratically. Under *Cauchy contamination*, the expected squared loss diverges for *any* centroid choice.

Solution—Hard thresholding: Assign weight $w^*(\mathbf{x}) = 1$ if $\|\mathbf{x} - \mathbf{c}\|^2 \leq \lambda$ and 0 otherwise. This excludes points beyond radius $\sqrt{\lambda}$ from the centroid, making the method robust to outliers.

6 Gaussian Models

6.1 Univariate Gaussian

The probability density function:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

The MLE for the parameters (derived in Section 7):

$$\hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^N x_i = \bar{x}, \quad \hat{\sigma}_{\text{MLE}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})^2$$

6.2 Multivariate Gaussian

Definition 6.1 (Multivariate Gaussian Distribution). For $\mathbf{x} \in \mathbb{R}^d$ with mean $\boldsymbol{\mu} \in \mathbb{R}^d$ and covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ (symmetric, positive definite):

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

where $|\boldsymbol{\Sigma}|$ denotes $\det(\boldsymbol{\Sigma})$ and $\boldsymbol{\Sigma}^{-1}$ is called the **precision matrix**.

Geometric interpretation: The quantity $(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$ is the squared **Mahalanobis distance** from $\mathbf{x}$ to $\boldsymbol{\mu}$. Level sets (curves of constant density) are *ellipses*:

- The axes of the ellipses are aligned with the **eigenvectors** of $\boldsymbol{\Sigma}$.
- The lengths of the axes are proportional to the **square roots of the eigenvalues** $\sqrt{\lambda_i}$.
- If $\boldsymbol{\Sigma} = \sigma^2 I$, the ellipses become circles (isotropic Gaussian).

Definition 6.2 (Covariance Matrix Properties). The covariance matrix $\boldsymbol{\Sigma}$ with $\Sigma_{ij} = \text{Cov}(X_i, X_j)$ is:

- **Symmetric:** $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^\top$ (since $\text{Cov}(X_i, X_j) = \text{Cov}(X_j, X_i)$).
- **Positive semi-definite:** $\mathbf{x}^\top \boldsymbol{\Sigma} \mathbf{x} \geq 0$ for all $\mathbf{x}$.
- Diagonal entries are variances: $\Sigma_{ii} = \text{Var}(X_i)$.
- Off-diagonal entries are covariances: $\Sigma_{ij} = \text{Cov}(X_i, X_j)$.

Key Idea 6.1 (Special Property of Multivariate Gaussians). For a multivariate Gaussian, **uncorrelated** $\Leftrightarrow$ **independent**:

$$\text{Cov}(X_i, X_j) = 0 \quad \Leftrightarrow \quad X_i \perp X_j$$

This is *not* true for general distributions (uncorrelated does not imply independent in general).

Moment Generating Function: $M_X(\mathbf{t}) = \exp(\mathbf{t}^\top \boldsymbol{\mu} + \frac{1}{2} \mathbf{t}^\top \boldsymbol{\Sigma} \mathbf{t})$

Linear transformation: If $X \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, then $\mathbf{a}^\top X \sim \mathcal{N}(\mathbf{a}^\top \boldsymbol{\mu}, \mathbf{a}^\top \boldsymbol{\Sigma} \mathbf{a})$. In fact, X is multivariate Gaussian if and only if every linear combination $\mathbf{a}^\top X$ is univariate Gaussian.

6.3 MLE for Multivariate Gaussian

Given n iid samples $\mathbf{x}_1, \dots, \mathbf{x}_n \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$:

$$\hat{\boldsymbol{\mu}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad \hat{\boldsymbol{\Sigma}} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^\top$$

Derivation sketch: The log-likelihood is:

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{n}{2} \log \det(\boldsymbol{\Sigma}) - \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) + \text{const}$$

Setting $\nabla_{\boldsymbol{\mu}} \ell = 0$ gives $\hat{\boldsymbol{\mu}} = \bar{\mathbf{x}}$. Setting $\nabla_{\boldsymbol{\Sigma}} \ell = 0$ (using $\nabla_A \log \det(A) = A^{-1}$ and $\nabla_A \text{Tr}(A^{-1}B) = -A^{-1}BA^{-1}$) gives $\hat{\boldsymbol{\Sigma}}$.

6.4 Mixture of Gaussians (GMM)

Definition 6.3 (Gaussian Mixture Model). A probability distribution defined as a weighted sum of K Gaussian components:

$$p(\mathbf{x} \mid \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

where $\pi_k \geq 0$ are the **mixing weights** (prior probabilities of each cluster) with $\sum_{k=1}^K \pi_k = 1$, and $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$.

Latent variable interpretation: Each data point $\mathbf{x}_i$ is generated by:

1. Sample a cluster assignment $z_i \sim \text{Multinomial}(\pi_1, \dots, \pi_K)$.
2. Sample data from that cluster: $\mathbf{x}_i \mid z_i = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

Log-likelihood:

$$\ell(\theta) = \sum_{i=1}^n \log \left[\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right]$$

The log-of-sum makes this **intractable** to optimize in closed form. This is why we need iterative methods like EM or gradient descent.

With known cluster assignments z : If we know which cluster each point belongs to, the MLE has a closed form:

$$\hat{\pi}_k = n_k / n \tag{7}$$

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i: z_i = k} \mathbf{x}_i \tag{8}$$

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{n_k} \sum_{i: z_i = k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top \tag{9}$$

Key Idea 6.2 (K-Means as a Special Case of GMM). K-means is the *zero-variance limit* of GMM:

- GMM: soft assignments (probability of belonging to each cluster).
- K-means: hard assignments (each point belongs to exactly one cluster).
- GMM: ellipsoidal clusters (different shapes allowed).
- K-means: spherical clusters of equal size.

GMM is strictly more general and makes its statistical assumptions explicit.

Uniform outlier component: Adding a uniform component $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}_k + \pi_{\text{out}}/V$ helps handle outliers. As a point moves far from all Gaussian centers, the Gaussian terms decay to 0 but the uniform term stays constant, so $\gamma_{\text{out}} \rightarrow 1$ —far-away points are automatically assigned to the outlier cluster.

7 Maximum Likelihood Estimation

7.1 The MLE Framework

Definition 7.1 (Maximum Likelihood Estimation (MLE)). Given iid data $D = \{x_1, \dots, x_N\}$ and a parametric model $p(x | \theta)$, the MLE finds the parameter that maximizes the likelihood of the observed data:

$$\hat{\theta}_{\text{MLE}} = \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^N p(x_i | \theta) = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^N \log p(x_i | \theta)$$

We take the log because: (1) products become sums (easier to differentiate), and (2) log is monotonically increasing so the maximizer is unchanged.

MLE Properties:

- **Consistency:** As $N \rightarrow \infty$, $\hat{\theta}_{\text{MLE}} \rightarrow \theta^*$ (the true parameter).
- **Efficiency:** Achieves the lowest variance among consistent estimators (asymptotically).
- **Invariance:** If $\hat{\theta}$ is the MLE for θ , then $g(\hat{\theta})$ is the MLE for $g(\theta)$.
- MLE gives a **point estimate**, not a distribution over parameters.

7.2 MLE Examples

Example 7.1 (MLE for Bernoulli). $X \sim \text{Bernoulli}(p)$. Given N trials with N_1 successes:

$$L(p) = p^{N_1} (1 - p)^{N_0}, \quad \ell(p) = N_1 \log p + N_0 \log(1 - p)$$

$$\frac{d\ell}{dp} = \frac{N_1}{p} - \frac{N_0}{1 - p} = 0 \implies \boxed{\hat{p}_{\text{MLE}} = \frac{N_1}{N}}$$

The MLE is simply the empirical frequency.

Example 7.2 (MLE for Poisson). $X \sim \text{Poisson}(\lambda)$. Log-likelihood: $\ell(\lambda) = \sum_i (x_i \log \lambda - \lambda) + \text{const.}$

$$\frac{d\ell}{d\lambda} = \frac{\sum x_i}{\lambda} - N = 0 \implies \boxed{\hat{\lambda}_{\text{MLE}} = \bar{x}}$$

Example 7.3 (MLE for Univariate Gaussian). Given $x_1, \dots, x_N \sim \mathcal{N}(\mu, \sigma^2)$:

$$\ell(\mu, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2$$

Setting $\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum (x_i - \mu) = 0$ gives $\hat{\mu} = \bar{x}$.

Setting $\frac{\partial \ell}{\partial \sigma^2} = 0$ gives $\hat{\sigma}^2 = \frac{1}{N} \sum (x_i - \hat{\mu})^2$.

Second derivative test: $\frac{\partial^2 \ell}{\partial \mu^2} = -N/\sigma^2 < 0$, confirming this is a maximum.

Example 7.4 (MLE for Multinomial (Lagrange Multipliers)). Given outcomes in $\{1, \dots, K\}$ with counts $n_1, \dots, n_K$ and probabilities $\theta_1, \dots, \theta_K$ where $\sum \theta_k = 1$:

$$\ell(\theta) = \sum_k n_k \log \theta_k, \quad \text{subject to } \sum_k \theta_k = 1$$

Using the Lagrangian $\mathcal{L} = \sum_k n_k \log \theta_k + \lambda(1 - \sum_k \theta_k)$:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{n_k}{\theta_k} - \lambda = 0 \implies \theta_k = \frac{n_k}{\lambda}$$

Substituting into the constraint: $\sum_k n_k / \lambda = 1 \implies \lambda = N$. So $\hat{\theta}_k = n_k / N$.

7.3 MAP Estimation

Definition 7.2 (Maximum A Posteriori (MAP) Estimation). MAP incorporates a prior $p(\theta)$ over the parameters:

$$\hat{\theta}_{\text{MAP}} = \underset{\theta}{\operatorname{argmax}} p(D | \theta) p(\theta) = \underset{\theta}{\operatorname{argmin}} [-\log p(D | \theta) - \log p(\theta)]$$

MAP = MLE + regularization from the prior. As $N \rightarrow \infty$, the data overwhelms the prior and MAP $\rightarrow$ MLE.

Key Idea 7.1 (Prior-Regularization Correspondence). The choice of prior determines the type of regularization:

Prior on $\mathbf{w}$	Regularization	Effect
Gaussian $\mathcal{N}(\mathbf{0}, \tau^2 I)$	L2 (Ridge): $\lambda \ \mathbf{w}\ _2^2$	Shrinks all weights
Laplace(0, b)	L1 (Lasso): $\lambda \ \mathbf{w}\ _1$	Zeros out some weights

Example 7.5 (Beta-Bernoulli Conjugacy). If the prior is $Y \sim \text{Beta}(\alpha, \beta)$ and we observe N_1 heads and N_0 tails:

$$\text{Posterior: } Y \mid D \sim \text{Beta}(\alpha + N_1, \beta + N_0) \quad (10)$$

$$\hat{Y}_{\text{MLE}} = \frac{N_1}{N} \quad (11)$$

$$\hat{Y}_{\text{MAP}} = \frac{N_1 + \alpha - 1}{N + \alpha + \beta - 2} \quad (12)$$

The posterior is the same family as the prior—this is **conjugacy**. The prior acts like “pseudo-observations”: $\alpha - 1$ prior heads, $\beta - 1$ prior tails. With large N , MAP approaches MLE.

8 Linear Regression

8.1 Problem Setup

Given data $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, linear regression models:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \quad (\text{or } \hat{y} = \mathbf{w}^\top \mathbf{x} + w_0 \text{ with bias})$$

Generative model: $Y = \mathbf{w}^\top \mathbf{x} + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$, so $p(y \mid \mathbf{x}, \mathbf{w}) = \mathcal{N}(y \mid \mathbf{w}^\top \mathbf{x}, \sigma^2)$.

Under MLE with Gaussian noise, maximizing the likelihood is equivalent to minimizing the sum of squared errors.

8.2 Ordinary Least Squares (OLS)

Definition 8.1 (OLS Objective). With design matrix $X \in \mathbb{R}^{N \times d}$ (rows are data points) and $\mathbf{y} \in \mathbb{R}^N$:

$$L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 = (\mathbf{y} - X\mathbf{w})^\top (\mathbf{y} - X\mathbf{w})$$

Expanding: $L = \mathbf{y}^\top \mathbf{y} - 2\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{w}^\top X^\top X \mathbf{w}$

Theorem 8.1 (Normal Equations and OLS Solution). Setting the gradient to zero:

$$\nabla_{\mathbf{w}} L = -2X^\top \mathbf{y} + 2X^\top X \mathbf{w} = \mathbf{0}$$

yields the **normal equations**:

$$X^\top X \mathbf{w} = X^\top \mathbf{y}$$

If $X^\top X$ is invertible (i.e., X has full column rank):

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$$

The Hessian is $H = 2X^\top X$, which is PD when X has full column rank, confirming this is a minimum.

8.3 Geometric Interpretation

Key Idea 8.1 (Projection View of OLS). The prediction $\hat{\mathbf{y}} = X\mathbf{w}^*$ is the **orthogonal projection** of $\mathbf{y}$ onto the column space of X .

- $\hat{\mathbf{y}}$ is the point in $\text{col}(X)$ closest to $\mathbf{y}$ (in Euclidean distance).
- The residual $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ is **orthogonal** to the column space: $X^\top \mathbf{r} = \mathbf{0}$.
- If X includes a column of all ones (bias term), then $\mathbf{1}^\top \mathbf{r} = 0$, meaning the residuals sum to zero.

8.4 When $X^\top X$ is Not Invertible

$X^\top X$ is not invertible when:

- Features are linearly dependent (multicollinearity).
- $d > N$ (more features than data points).

Solutions: regularization (Ridge, Lasso) or feature selection.

8.5 Ridge Regression (L2 Regularization)

Definition 8.2 (Ridge Regression). Add an L2 penalty on the weights:

$$L_{\text{ridge}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\lambda > 0$ is the regularization parameter.

Solution (closed form):

$$\mathbf{w}_{\text{ridge}}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

Derivation:

$$\begin{aligned} \nabla_{\mathbf{w}} L_{\text{ridge}} &= -2X^\top \mathbf{y} + 2X^\top X \mathbf{w} + 2\lambda \mathbf{w} = \mathbf{0} \\ (X^\top X + \lambda I) \mathbf{w} &= X^\top \mathbf{y} \end{aligned}$$

Key Idea 8.2 (Properties of Ridge Regression). • **Always invertible:** $X^\top X + \lambda I$ is PD for $\lambda > 0$, even when $X^\top X$ is singular.

- **Shrinkage:** All weights are shrunk toward zero, but none are set exactly to zero.
- **Condition number improvement:** $\kappa_{\text{ridge}} = \frac{\sigma_{\max}^2 + \lambda}{\sigma_{\min}^2 + \lambda} < \kappa_{\text{OLS}}$.
- **Bayesian interpretation:** Ridge = MAP with Gaussian prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 I)$, where $\lambda = \sigma^2 / \tau^2$.
- As $\lambda \rightarrow 0$: approaches OLS. As $\lambda \rightarrow \infty$: $\mathbf{w} \rightarrow \mathbf{0}$.

8.6 Lasso Regression (L1 Regularization)

Definition 8.3 (Lasso Regression). Add an L1 penalty on the weights:

$$L_{\text{lasso}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = \sum_j |w_j|$.

Key properties:

- **No closed-form solution** (L1 norm is not differentiable at 0).
- **Induces sparsity:** Some w_i are set exactly to zero $\rightarrow$ automatic feature selection.
- **Bayesian interpretation:** MAP with Laplace prior, $\lambda = 2\sigma^2/b$.

Key Idea 8.3 (Why L1 Promotes Sparsity but L2 Does Not). When $X^\top X = nI$ (orthogonal features), each weight can be optimized independently:

- **L1:** $w_i^* = 0$ when $|k_i| \leq \lambda/2$. This is an *interval* condition—many values of k_i lead to exact zeros.
- **L2:** $w_i^* = 0$ only when $k_i = 0$ exactly. This is a *single point*—probability zero for continuous data.

Geometric intuition: The L1 constraint surface is a diamond (has corners on the axes). The L2 constraint surface is a circle (smooth everywhere). When elliptical loss contours are tangent to the constraint surface:

- Diamond corners lie on the axes (where some $w_i = 0$) $\rightarrow$ tangency at corners is likely $\rightarrow$ sparse solutions.
- Circle has no corners $\rightarrow$ tangency typically occurs at points where no $w_i = 0$.

8.7 Elastic Net

Combines L1 and L2:

$$L_{\text{EN}}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2$$

Benefits: sparsity from L1 + stability for correlated features from L2. Lasso tends to arbitrarily pick one of a group of correlated features; Elastic Net handles groups better.

8.8 Weighted Least Squares

Definition 8.4 (Weighted Least Squares (WLS)). Different weights for different data points:

$$L(\mathbf{w}) = (\mathbf{y} - X\mathbf{w})^\top A (\mathbf{y} - X\mathbf{w}), \quad A = \text{diag}(\alpha_1, \dots, \alpha_n)$$

Solution: $\mathbf{w}^* = (X^\top A X)^{-1} X^\top A \mathbf{y}$

Invertible when X has full column rank and all $\alpha_i > 0$.

8.9 Regression Metrics

- **MSE:** $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$ (mean squared error)
- **RMSE:** $\sqrt{\text{MSE}}$ (in same units as y)
- **MAE:** $\frac{1}{n} \sum |y_i - \hat{y}_i|$ (more robust to outliers)
- **R^2 :** $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ (fraction of variance explained; $R^2 = 1$ is perfect)

Key Idea 8.4 (L1 vs L2 Loss Minimizers). • **L2 loss** $\sum (y_i - c)^2$ is minimized by the **mean**: $c^* = \bar{y}$.

• **L1 loss** $\sum |y_i - c|$ is minimized by the **median**: $c^* = \text{median}(y_1, \dots, y_n)$.
The median is more robust to outliers than the mean.

9 Bias-Variance Tradeoff

9.1 Setup

Assume the true data-generating process is $t = h(\mathbf{x}) + \epsilon$ where $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2$ (the noise is independent of $\mathbf{x}$ and the training data D). We train a model $f_D(\mathbf{x})$ on a random training set D . The expected prediction error at a test point $\mathbf{x}$ is:

Theorem 9.1 (Bias-Variance Decomposition).

$$\mathbb{E}_{D, \epsilon} [(t - f_D(\mathbf{x}))^2] = \underbrace{\sigma^2}_{\text{noise (irreducible)}} + \underbrace{(h(\mathbf{x}) - \mathbb{E}_D[f_D(\mathbf{x})])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D [(f_D(\mathbf{x}) - \mathbb{E}_D[f_D(\mathbf{x})])^2]}_{\text{model variance}}$$

where the expectation $\mathbb{E}_D$ is over the randomness in the training data (different training sets give different learned models).

9.2 Full Proof of the Decomposition

Theorem 9.2 (Bias-Variance Decomposition Proof). Let $\bar{f}(\mathbf{x}) = \mathbb{E}_D[f_D(\mathbf{x})]$ denote the “average model” (averaged over all possible training sets).

Step 1: Decompose the error into three terms.

$$t - f_D(\mathbf{x}) = (t - h(\mathbf{x})) + (h(\mathbf{x}) - \bar{f}(\mathbf{x})) + (\bar{f}(\mathbf{x}) - f_D(\mathbf{x})) \quad (13)$$

$$= \underbrace{\epsilon}_{\text{noise}} + \underbrace{(h(\mathbf{x}) - \bar{f}(\mathbf{x}))}_{\text{bias}} + \underbrace{(\bar{f}(\mathbf{x}) - f_D(\mathbf{x}))}_{\text{deviation from mean model}} \quad (14)$$

Step 2: Square and expand.

$$(t - f_D)^2 = \epsilon^2 + (h - \bar{f})^2 + (\bar{f} - f_D)^2 + 2\epsilon(h - \bar{f}) + 2\epsilon(\bar{f} - f_D) + 2(h - \bar{f})(\bar{f} - f_D)$$

Step 3: Take expectation $\mathbb{E}_{D,\epsilon}$ and show all cross terms vanish.

Cross term 1: $\mathbb{E}[2\epsilon(h - \bar{f})] = 2(h - \bar{f})\mathbb{E}[\epsilon] = 0$ since $\mathbb{E}[\epsilon] = 0$.

Cross term 2: $\mathbb{E}[2\epsilon(\bar{f} - f_D)] = 2\mathbb{E}_\epsilon[\epsilon] \cdot \mathbb{E}_D[\bar{f} - f_D] = 0$ since ϵ is independent of D and $\mathbb{E}[\epsilon] = 0$.

Cross term 3: $\mathbb{E}_D[2(h - \bar{f})(\bar{f} - f_D)] = 2(h - \bar{f})\mathbb{E}_D[\bar{f} - f_D] = 2(h - \bar{f}) \cdot 0 = 0$ since $\mathbb{E}_D[f_D] = \bar{f}$ by definition.

Step 4: Remaining terms.

$$\mathbb{E}[(t - f_D)^2] = \underbrace{\mathbb{E}[\epsilon^2]}_{=\sigma^2} + \underbrace{(h(\mathbf{x}) - \bar{f}(\mathbf{x}))^2}_{=\text{bias}^2} + \underbrace{\mathbb{E}_D[(\bar{f}(\mathbf{x}) - f_D(\mathbf{x}))^2]}_{=\text{variance}} \quad \blacksquare \quad (15)$$

9.3 Interpretation

Definition 9.1 (The Three Error Components). • **Noise** (σ^2): Irreducible error inherent in the data. Even the true function $h(\mathbf{x})$ cannot predict t perfectly because of random noise. No model can reduce this.

- **Bias**² = $(h(\mathbf{x}) - \bar{f}(\mathbf{x}))^2$: Measures how far the *average model* $\bar{f}$ is from the truth h . If you could train on infinitely many datasets and average the models, would you get the right answer?

- High bias $\rightarrow$ the model family can’t capture the true function $\rightarrow$ **underfitting**.
- Example: fitting a line to quadratic data. No matter how much data you give, the average model is still a line.

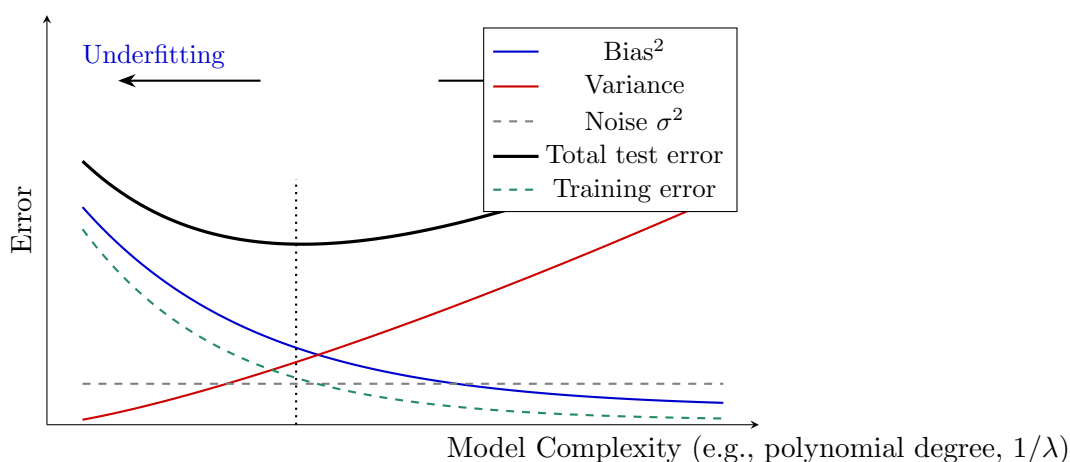
- **Variance** = $\mathbb{E}_D[(f_D(\mathbf{x}) - \bar{f}(\mathbf{x}))^2]$: Measures how much the learned model *varies* across different training sets. How sensitive is the model to the particular data it sees?

- High variance $\rightarrow$ the model changes dramatically with different training data $\rightarrow$ **overfitting**.
- Example: fitting a degree-20 polynomial to 10 data points. Small changes in data drastically change the polynomial.

Example 9.1 (Bias-Variance Intuition: Dartboard Analogy). Imagine throwing darts at a target (the truth $h(\mathbf{x})$):

- **Low bias, low variance:** Darts clustered tightly around the bullseye. (Ideal!)
- **Low bias, high variance:** Darts centered on the bullseye but scattered widely. (Overfitting: right on average, but individual predictions are unreliable.)
- **High bias, low variance:** Darts clustered tightly but off-center. (Underfitting: consistently wrong in the same direction.)
- **High bias, high variance:** Darts scattered and off-center. (Worst case.)

9.4 The Tradeoff



Key Idea 9.1 (Regularization Controls the Bias-Variance Tradeoff). • **Increasing λ** (more regularization): weights shrink toward zero, model becomes simpler → **higher bias** (model is too constrained to capture the true function), **lower variance** (model changes less with different training data) → **underfitting**.

- **Decreasing λ** (less regularization): model becomes more flexible → **lower bias** (can fit complex patterns), **higher variance** (more sensitive to training data noise) → **overfitting**.

- **Optimal λ :** minimizes the total test error (the “sweet spot”). Found via cross-validation.

Other factors that affect the tradeoff:

- **More training data N :** Reduces variance (more data stabilizes the model) without increasing bias. Allows using more complex models without overfitting.
- **Higher-degree features:** Increases model complexity → lower bias but higher variance.
- **Ensemble methods:** Averaging multiple models reduces variance while keeping bias roughly the same.

Example 9.2 (Bias-Variance for Ridge Regression). For Ridge regression with parameter λ , the weight vector is:

$$\mathbf{w}_\lambda = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

- **Bias:** $\text{Bias}(\mathbf{w}_\lambda) = \mathbb{E}[\mathbf{w}_\lambda] - \mathbf{w}^* = -\lambda(X^\top X + \lambda I)^{-1} \mathbf{w}^*$. Bias is **0** when $\lambda = 0$ (OLS) and grows with λ .
- **Variance:** $\text{Var}(\hat{y}) = \sigma^2 \sum_j \frac{\sigma_j^2}{(\sigma_j^2 + \lambda)^2}$ where σ_j are singular values of X . Variance decreases as λ increases.
- **Optimal λ :** Minimizes $\text{Bias}^2 + \text{Variance}$. Always exists and is > 0 (Ridge always improves over OLS in terms of expected test error).

9.5 K-Fold Cross-Validation

Definition 9.2 (K-Fold Cross-Validation). 1. Split training data into K equal-sized folds $F_1, \dots, F_K$.

2. For each fold $k = 1, \dots, K$:

- Train on all data *except* fold k : $D_{\text{train}} = D \setminus F_k$.
- Evaluate on fold k : $\text{err}_k = \frac{1}{|F_k|} \sum_{i \in F_k} L(y_i, f_{D_{\text{train}}}(\mathbf{x}_i))$.

3. **CV error:** $\text{CV}(\lambda) = \frac{1}{K} \sum_{k=1}^K \text{err}_k$

Select: $\lambda^* = \text{argmin}_\lambda \text{CV}(\lambda)$, then retrain on ALL data with λ^* .

Common choices: $K = 5$ or $K = 10$. $K = N$ is “Leave-One-Out” (LOO) CV — nearly unbiased but high variance.

Key Idea 9.2 (Why Cross-Validation Works). • **Training error** is a biased (optimistic) estimate of test error — the model was fit to this data!

- Each CV fold provides an *unbiased* estimate of test error (the validation fold was not used for training).
- Averaging K estimates reduces variance.
- **Tradeoff in K :** Larger K uses more training data per fold (less bias in error estimate) but folds are more similar (higher variance of the estimate, more computation).

10 Classification

10.1 Three Approaches to Classification

1. **Generative:** Model the class-conditional densities $p(\mathbf{x} \mid Y = k)$ and priors $p(Y = k)$, then use Bayes’ rule:

$$p(Y = k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid Y = k) p(Y = k)}{p(\mathbf{x})}$$

Examples: GDA (LDA, QDA), Naive Bayes.

2. **Discriminative:** Model $p(Y | \mathbf{x})$ directly without modeling $p(\mathbf{x})$. Example: Logistic Regression.
3. **Decision boundary:** Learn $f : \mathbf{x} \rightarrow \{1, \dots, K\}$ directly. Example: Perceptron, SVM.

Generative vs. Discriminative: Generative models make stronger assumptions; when those assumptions are correct, they can be more data-efficient. Discriminative models are more robust to model misspecification.

10.2 Gaussian Discriminant Analysis (GDA)

Definition 10.1 (GDA Setup). Assume each class k has a Gaussian class-conditional distribution:

$$p(\mathbf{x} | Y = k) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

with class prior $p(Y = k) = \pi_k$.

Classification rule (Bayes optimal): Predict class k^* that maximizes the posterior:

$$k^* = \operatorname{argmax}_k p(Y = k | \mathbf{x}) = \operatorname{argmax}_k p(\mathbf{x} | Y = k) \pi_k$$

Since $p(\mathbf{x})$ is the same for all classes, we can ignore it.

10.2.1 MLE for GDA Parameters

Given labeled data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $n_k = |\{i : y_i = k\}|$:

Theorem 10.1 (MLE for GDA).

$$\hat{\pi}_k = \frac{n_k}{n} \quad (\text{class proportion — Multinomial MLE}) \quad (16)$$

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i:y_i=k} \mathbf{x}_i \quad (\text{class mean — Gaussian MLE}) \quad (17)$$

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{n_k} \sum_{i:y_i=k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top \quad (\text{class covariance}) \quad (18)$$

Derivation of $\hat{\boldsymbol{\mu}}_k$: The log-likelihood for class k data is:

$$\ell_k(\boldsymbol{\mu}_k) = -\frac{1}{2} \sum_{i:y_i=k} (\mathbf{x}_i - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_k) + \text{const}$$

Taking the gradient and setting to zero:

$$\nabla_{\boldsymbol{\mu}_k} \ell_k = \boldsymbol{\Sigma}_k^{-1} \sum_{i:y_i=k} (\mathbf{x}_i - \boldsymbol{\mu}_k) = \mathbf{0} \implies \hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i:y_i=k} \mathbf{x}_i$$

(The $\boldsymbol{\Sigma}_k^{-1}$ factor drops out since it multiplies zero.)

Shared covariance MLE (for LDA): When we assume $\boldsymbol{\Sigma}_A = \boldsymbol{\Sigma}_B = \boldsymbol{\Sigma}$, the MLE pools data from both classes:

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{n} \sum_k \sum_{i:y_i=k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top = \frac{n_A}{n} \hat{\boldsymbol{\Sigma}}_A + \frac{n_B}{n} \hat{\boldsymbol{\Sigma}}_B$$

This is the **weighted average** of the per-class covariances.

10.2.2 Deriving the Decision Boundary

The decision boundary is where $p(Y = A \mid \mathbf{x}) = p(Y = B \mid \mathbf{x})$, equivalently where the **log-posterior ratio** is zero. We classify A when:

$$\log \frac{p(Y = A \mid \mathbf{x})}{p(Y = B \mid \mathbf{x})} = \log \frac{p(\mathbf{x} \mid A)}{p(\mathbf{x} \mid B)} + \log \frac{\pi_A}{\pi_B} > 0$$

Substituting the Gaussian PDFs:

$$\begin{aligned} \log \frac{p(\mathbf{x} \mid A)}{p(\mathbf{x} \mid B)} &= -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_A)^\top \boldsymbol{\Sigma}_A^{-1} (\mathbf{x} - \boldsymbol{\mu}_A) + \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_B)^\top \boldsymbol{\Sigma}_B^{-1} (\mathbf{x} - \boldsymbol{\mu}_B) \\ &\quad - \frac{1}{2} \log \frac{|\boldsymbol{\Sigma}_A|}{|\boldsymbol{\Sigma}_B|} \end{aligned} \quad (19)$$

Theorem 10.2 (QDA Decision Boundary (General Case: $\Sigma_A \neq \Sigma_B$)). Expanding the quadratic forms in (19), the decision boundary is:

$$\mathbf{x}^\top \underbrace{\frac{1}{2}(\Sigma_B^{-1} - \Sigma_A^{-1})}_{\text{quadratic term}} \mathbf{x} + \underbrace{(\Sigma_A^{-1} \mu_A - \Sigma_B^{-1} \mu_B)^\top}_{\text{linear term}} \mathbf{x} + c = 0$$

where c collects all constant terms (involving $\mu_k^\top \Sigma_k^{-1} \mu_k$, $\log |\Sigma_k|$, and $\log \pi_k$).

This is a **quadratic function** of $\mathbf{x}$ — the boundary is a conic section (ellipse, parabola, or hyperbola).

Theorem 10.3 (LDA Decision Boundary (Shared Covariance: $\Sigma_A = \Sigma_B = \Sigma$)). When covariances are equal, the quadratic terms **cancel exactly**:

$$\mathbf{x}^\top \frac{1}{2}(\Sigma^{-1} - \Sigma^{-1})\mathbf{x} = 0$$

Also, $\log |\Sigma_A|/|\Sigma_B| = 0$. What remains is:

$$\begin{aligned} & -\frac{1}{2}\mathbf{x}^\top \Sigma^{-1}\mathbf{x} + \mu_A^\top \Sigma^{-1}\mathbf{x} - \frac{1}{2}\mu_A^\top \Sigma^{-1}\mu_A \\ & + \frac{1}{2}\mathbf{x}^\top \Sigma^{-1}\mathbf{x} - \mu_B^\top \Sigma^{-1}\mathbf{x} + \frac{1}{2}\mu_B^\top \Sigma^{-1}\mu_B + \log \frac{\pi_A}{\pi_B} \end{aligned} \quad (20)$$

The $\mathbf{x}^\top \Sigma^{-1}\mathbf{x}$ terms cancel, leaving:

$$\boxed{\underbrace{(\mu_A - \mu_B)^\top \Sigma^{-1}}_{\mathbf{w}^\top} \mathbf{x} + \underbrace{\left(-\frac{1}{2}\mu_A^\top \Sigma^{-1}\mu_A + \frac{1}{2}\mu_B^\top \Sigma^{-1}\mu_B + \log \frac{\pi_A}{\pi_B}\right)}_b = 0}$$

This is **linear** in $\mathbf{x}$! The weight vector is $\mathbf{w} = \Sigma^{-1}(\mu_A - \mu_B)$.

10.3 LDA vs QDA: The Four Cases

Theorem 10.4 (Decision Boundary Cases). **Case 1:** $\Sigma_A = \Sigma_B = \sigma^2 I$ (isotropic, equal). Equal priors ($\pi_A = \pi_B$).

- $\mathbf{w} = (\sigma^2 I)^{-1}(\boldsymbol{\mu}_A - \boldsymbol{\mu}_B) = \frac{1}{\sigma^2}(\boldsymbol{\mu}_A - \boldsymbol{\mu}_B)$, proportional to the line connecting the means.
- Bias: $b = -\frac{1}{2\sigma^2}(\|\boldsymbol{\mu}_A\|^2 - \|\boldsymbol{\mu}_B\|^2)$, i.e., $\|\mathbf{x} - \boldsymbol{\mu}_A\|^2 = \|\mathbf{x} - \boldsymbol{\mu}_B\|^2$.
- Decision boundary: **perpendicular bisector** of the segment $\boldsymbol{\mu}_A \boldsymbol{\mu}_B$. **Linear.**

Case 2: $\Sigma_A = \Sigma_B = \Sigma$ (shared, non-isotropic). Equal priors.

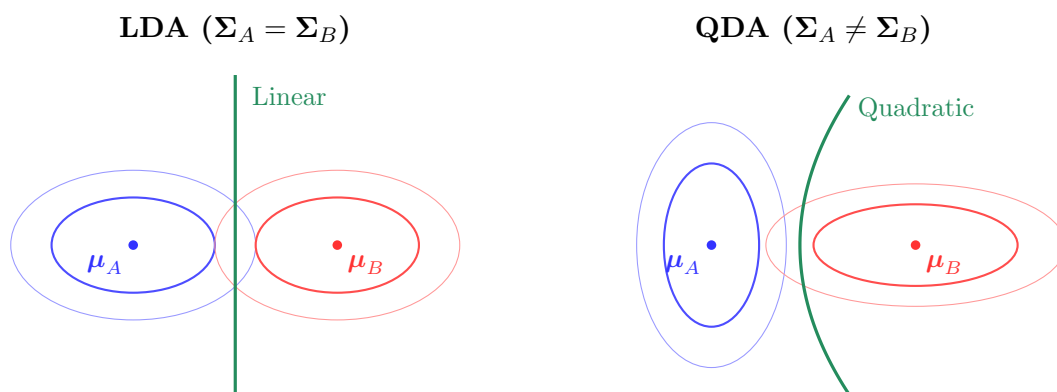
- Mahalanobis distances equal: $(\mathbf{x} - \boldsymbol{\mu}_A)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_A) = (\mathbf{x} - \boldsymbol{\mu}_B)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_B)$.
- **Linear** boundary, but tilted by Σ^{-1} (not necessarily perpendicular to $\boldsymbol{\mu}_A - \boldsymbol{\mu}_B$).
- **Whitening:** Apply $\hat{\mathbf{x}} = \Sigma^{-1/2}\mathbf{x}$. In the whitened space, both classes have $\hat{\Sigma} = I$, reducing to Case 1.

Case 3: $\Sigma_A = \Sigma_B$ but $\pi_A \neq \pi_B$ (unequal priors).

- The $\log(\pi_A/\pi_B)$ term shifts $b \rightarrow$ boundary shifts toward the **less probable** class. Still **linear**.
- **Intuition:** If class A is much more common, we should predict A unless the evidence for B is strong, so the decision boundary moves away from A 's mean.

Case 4: $\Sigma_A \neq \Sigma_B$ (different covariances).

- The quadratic terms $\mathbf{x}^\top (\Sigma_B^{-1} - \Sigma_A^{-1})\mathbf{x}$ **do not cancel** $\rightarrow$ **quadratic boundary** (QDA).
- Decision boundary is a conic section: can be an ellipse, parabola, or hyperbola depending on the eigenvalues of $\Sigma_B^{-1} - \Sigma_A^{-1}$.



Definition 10.2 (LDA (Linear Discriminant Analysis)). • Assumes shared covariance: $\Sigma_A = \Sigma_B = \Sigma$.

- Decision boundary is **linear**: $\mathbf{w}^\top \mathbf{x} + b = 0$ where $\mathbf{w} = \Sigma^{-1}(\mu_A - \mu_B)$.
- The predictive distribution has a **sigmoid** form: $p(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$.
- Parameters to estimate: K means + 1 shared covariance matrix = $Kd + d(d+1)/2$.
- **When to use**: When you believe classes have similar spread/shape but different centers.

Example 10.1 (LDA Posterior is Sigmoid). Starting from the log-posterior ratio for two classes with shared Σ :

$$\log \frac{p(Y = 1 \mid \mathbf{x})}{p(Y = 0 \mid \mathbf{x})} = \underbrace{(\mu_1 - \mu_0)^\top \Sigma^{-1}}_{\mathbf{w}^\top} \mathbf{x} + \underbrace{\text{const}}_b = \mathbf{w}^\top \mathbf{x} + b$$

Now, if we let $a = \mathbf{w}^\top \mathbf{x} + b$, then:

$$\frac{p(Y = 1 \mid \mathbf{x})}{1 - p(Y = 1 \mid \mathbf{x})} = e^a \implies p(Y = 1 \mid \mathbf{x}) = \frac{e^a}{1 + e^a} = \frac{1}{1 + e^{-a}} = \sigma(a)$$

This is exactly the **sigmoid function** — LDA's posterior is a logistic regression model!

Definition 10.3 (QDA (Quadratic Discriminant Analysis)). • Each class has its own covariance: $\Sigma_A \neq \Sigma_B$.

- Decision boundary is **quadratic** (conic section: ellipse, parabola, or hyperbola).
- More flexible but requires more parameters: K means + K covariances = $Kd + Kd(d+1)/2$.
- **Risk of overfitting**: With $d = 100$ and $K = 2$, QDA needs $\sim 10,000$ covariance parameters vs. $\sim 5,000$ for LDA.
- **When to use**: When classes clearly have different shapes/spreads (e.g., one class is compact, another is spread out).

Key Idea 10.1 (LDA $\subset$ Logistic Regression). LDA always produces a decision boundary of the form $\sigma(\mathbf{w}^\top \mathbf{x} + b)$, which is exactly the logistic regression model. So every LDA model is a logistic regression model, but not vice versa.

LDA makes the additional assumption that $p(\mathbf{x} \mid Y)$ is Gaussian with shared covariance—this is a stronger (and possibly wrong) assumption. Logistic regression only assumes the log-odds are linear in $\mathbf{x}$.

Tradeoff:

- **LDA advantage**: If the Gaussian assumption is correct, LDA is more data-efficient (fewer effective parameters to learn from data).
- **Logistic regression advantage**: More robust if the Gaussian assumption is wrong. Only models the boundary directly.

Definition 10.4 (Whitening Transformation). Given data with covariance $\Sigma = U S U^\top$ (eigen-decomposition), the whitening transform is:

$$\hat{\mathbf{x}} = S^{-1/2} U^\top \mathbf{x}$$

This transforms the data so that $\hat{\Sigma} = I$ (identity covariance). Geometrically, whitening:

1. Rotates data to align with eigenvectors ($U^\top$)
2. Scales each direction by $1/\sqrt{\lambda_i}$ ($S^{-1/2}$)

After whitening, Mahalanobis distance becomes Euclidean distance, and LDA reduces to the perpendicular bisector case.

10.4 Naive Bayes

Definition 10.5 (Naive Bayes). Assume features are conditionally independent given the class:

$$p(\mathbf{x} \mid Y = k) = \prod_{f=1}^d p(x_f \mid Y = k)$$

This is equivalent to assuming a **diagonal covariance matrix** in GDA.

Despite the “naive” independence assumption (which is usually wrong), Naive Bayes often works surprisingly well in practice. It’s fast ($O(nD)$) and requires estimating only d parameters per class instead of $d(d+1)/2$.

10.5 Logistic Regression

10.5.1 The Sigmoid Function

Definition 10.6 (Sigmoid Function).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Key properties:

- $\sigma(z) \in (0, 1)$ for all $z \in \mathbb{R}$. Converts any real number to a probability.
- $\sigma(0) = 1/2$. Monotonically increasing.
- **Symmetry:** $\sigma(-z) = 1 - \sigma(z)$
- **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (elegant and easy to compute!)
- **Odds ratio:** $\frac{\sigma(z)}{1 - \sigma(z)} = e^z$, so $\log \frac{\sigma(z)}{1 - \sigma(z)} = z$

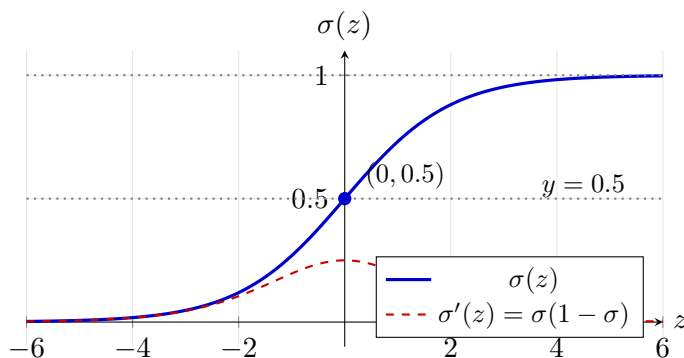
Example 10.2 (Sigmoid Derivative Proof). Starting from $\sigma(z) = (1 + e^{-z})^{-1}$:

$$\sigma'(z) = -(-e^{-z})(1 + e^{-z})^{-2} = \frac{e^{-z}}{(1 + e^{-z})^2} \quad (21)$$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \quad (22)$$

$$= \sigma(z) \cdot (1 - \sigma(z)) \quad (23)$$

This is a parabola in σ , maximized at $\sigma = 1/2$ (i.e., $z = 0$) where $\sigma' = 1/4$.



10.5.2 Logistic Regression Model

Definition 10.7 (Binary Logistic Regression).

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

$$P(Y = 0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x}) = \sigma(-\mathbf{w}^\top \mathbf{x})$$

The **log-odds** (logit) are linear in $\mathbf{x}$:

$$\log \frac{P(Y = 1 \mid \mathbf{x})}{P(Y = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

Decision boundary: $\mathbf{w}^\top \mathbf{x} = 0$ (a linear hyperplane, where $P(Y = 1) = 0.5$).

Key Idea 10.2 (Why Sigmoid? (Intuition for Logistic Regression)). **Reason 1 — From LDA:** As shown above, if class-conditionals are Gaussian with shared covariance, the posterior $p(Y = 1 \mid \mathbf{x})$ is exactly a sigmoid of a linear function. Logistic regression *directly models* this form without assuming Gaussianity.

Reason 2 — Log-odds linearity: We want to model the probability of class 1 as a function of $\mathbf{x}$. The simplest model is $\log \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x}$ (log-odds linear in features). Solving for p gives $p = \sigma(\mathbf{w}^\top \mathbf{x})$.

Reason 3 — Exponential family: The Bernoulli distribution is in the exponential family. The sigmoid is its *canonical link function*, connecting the natural parameter to the mean. This makes the MLE well-behaved (concave log-likelihood).

Reason 4 — Maximum entropy: Among all distributions on $\{0, 1\}$ with $\mathbb{E}[\mathbf{x} \mid Y]$ matching the data, the one with maximum entropy is the logistic model.

Key Idea 10.3 (Discriminative vs Generative: Logistic Regression vs LDA).

	LDA (Generative)	Logistic Reg. (Discrim.)
Models	$p(\mathbf{x} Y)$ and $p(Y)$	$p(Y \mathbf{x})$ directly
Assumption	Gaussian class-conditionals	Log-odds linear in $\mathbf{x}$
Boundary	$\mathbf{w} = \Sigma^{-1}(\mu_1 - \mu_0)$	Learned by optimization
Data efficiency	Better if Gaussian correct	Better if assumption wrong
Fit method	Closed-form MLE	Iterative (gradient descent)

10.5.3 MLE for Logistic Regression (Full Derivation)

Theorem 10.5 (Logistic Regression MLE). The likelihood for n data points with $y_i \in \{0, 1\}$:

$$L(\mathbf{w}) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}, \quad p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

Step 1: Log-likelihood. Take the log:

$$\ell(\mathbf{w}) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

This is the **negative binary cross-entropy** (up to sign). Maximizing ℓ = minimizing cross-entropy.

Step 2: Rewrite using sigmoid symmetry. Since $1 - \sigma(z) = \sigma(-z)$ and $\log \sigma(z) = -\log(1 + e^{-z})$:

$$\ell(\mathbf{w}) = \sum_{i=1}^n \left[-y_i \log(1 + e^{-\mathbf{w}^\top \mathbf{x}_i}) - (1 - y_i) \log(1 + e^{\mathbf{w}^\top \mathbf{x}_i}) \right]$$

Or equivalently with $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$: $\ell = -\sum_i \log(1 + e^{-\tilde{y}_i \mathbf{w}^\top \mathbf{x}_i})$.

Theorem 10.6 (Gradient of Logistic Regression (Detailed Derivation)). For a single data point (x_i, y_i) , let $z_i = \mathbf{w}^\top \mathbf{x}_i$ and $p_i = \sigma(z_i)$:

$$\ell_i = y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i)) \quad (24)$$

$$\frac{\partial \ell_i}{\partial \mathbf{w}} = y_i \cdot \frac{\sigma'(z_i)}{\sigma(z_i)} \cdot \mathbf{x}_i + (1 - y_i) \cdot \frac{-\sigma'(z_i)}{1 - \sigma(z_i)} \cdot \mathbf{x}_i \quad (25)$$

Now use $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$= y_i \cdot \frac{\sigma(z_i)(1 - \sigma(z_i))}{\sigma(z_i)} \cdot \mathbf{x}_i - (1 - y_i) \cdot \frac{\sigma(z_i)(1 - \sigma(z_i))}{1 - \sigma(z_i)} \cdot \mathbf{x}_i \quad (26)$$

$$= y_i(1 - p_i)\mathbf{x}_i - (1 - y_i)p_i\mathbf{x}_i \quad (27)$$

$$= (y_i - y_i p_i - p_i + y_i p_i)\mathbf{x}_i = \boxed{(y_i - p_i)\mathbf{x}_i} \quad (28)$$

Summing over all points:

$$\nabla_{\mathbf{w}} \ell = \sum_{i=1}^n (y_i - p_i)\mathbf{x}_i = X^\top (\mathbf{y} - \mathbf{p})$$

where $\mathbf{p} = [\sigma(\mathbf{w}^\top \mathbf{x}_1), \dots, \sigma(\mathbf{w}^\top \mathbf{x}_n)]^\top$.

Interpretation: The gradient is a sum of feature vectors $\mathbf{x}_i$, each weighted by the *residual* $y_i - p_i$. Points where the prediction is wrong contribute more.

Theorem 10.7 (Hessian of Logistic Regression (Proves Concavity)). The Hessian of the log-likelihood:

$$\frac{\partial^2 \ell}{\partial \mathbf{w} \partial \mathbf{w}^\top} = - \sum_{i=1}^n p_i(1 - p_i)\mathbf{x}_i \mathbf{x}_i^\top = -X^\top D X \quad (29)$$

where $D = \text{diag}(p_1(1 - p_1), \dots, p_n(1 - p_n))$ is a diagonal matrix with $D_{ii} = p_i(1 - p_i) > 0$.

Proof that H is negative semi-definite: For any $\mathbf{v} \neq \mathbf{0}$:

$$\mathbf{v}^\top H \mathbf{v} = - \sum_i p_i(1 - p_i)(\mathbf{x}_i^\top \mathbf{v})^2 \leq 0$$

since each term is non-negative. Therefore, $\ell(\mathbf{w})$ is **concave**, so any local max is a global max. There is **no risk of getting stuck in local optima**.

Note: The Hessian depends on $\mathbf{w}$ (through p_i), so the loss surface is not quadratic (unlike OLS). But concavity guarantees that gradient descent converges to the global optimum.

Definition 10.8 (Gradient Ascent for Logistic Regression). Since we maximize $\ell(\mathbf{w})$ (which is concave), gradient *ascent* converges to the global optimum:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \cdot X^\top (\mathbf{y} - \mathbf{p}^{(t)})$$

Equivalently, minimizing the **negative log-likelihood** (NLL) via gradient descent:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \cdot X^\top (\mathbf{p}^{(t)} - \mathbf{y})$$

Example 10.3 (Regularized Logistic Regression). Add an L2 penalty for MAP estimation (Gaussian prior on $\mathbf{w}$):

$$\ell_{\text{reg}}(\mathbf{w}) = \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)] - \lambda \|\mathbf{w}\|_2^2$$

Gradient: $\nabla_{\mathbf{w}} \ell_{\text{reg}} = X^\top (\mathbf{y} - \mathbf{p}) - 2\lambda \mathbf{w}$

This prevents weights from growing unboundedly (which happens when data is linearly separable — the MLE pushes $\|\mathbf{w}\| \rightarrow \infty$ to make predictions more confident).

10.5.4 Multi-Class: Softmax Regression

Definition 10.9 (Softmax / Multinomial Logistic Regression). For K classes with weight vectors $\mathbf{w}_1, \dots, \mathbf{w}_K$:

$$P(Y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Loss: Multi-class cross-entropy: $-\sum_{i=1}^n \sum_{k=1}^K \mathbf{1}[y_i = k] \log P(Y = k \mid \mathbf{x}_i)$

Properties:

- Reduces to sigmoid for $K = 2$.
- Invariant to adding a constant to all scores (i.e., $\mathbf{w}_k^\top \mathbf{x} + c$ gives the same probabilities).

Label smoothing: Instead of hard one-hot labels $[1, 0, 0]$, use soft labels $[0.9, 0.05, 0.05]$. This prevents overconfident predictions and improves generalization.

10.6 Evaluating Classifiers

10.6.1 Confusion Matrix and Metrics

Definition 10.10 (Confusion Matrix). For binary classification with labels $\{-, +\}$:

	Predicted $-$	Predicted $+$
Actual $-$	TN (correct rejection)	FP (Type I error)
Actual $+$	FN (Type II error)	TP (hit)

Note: $\text{TN} + \text{FP} = \text{total actual negatives}$; $\text{FN} + \text{TP} = \text{total actual positives}$.

Definition 10.11 (Classification Metrics (Detailed)). **Rates computed from actual positives:**

- **TPR (True Positive Rate / Recall / Sensitivity):** $\frac{TP}{TP + FN}$

“Of all actual positives, what fraction did we catch?” Used when *missing a positive* is costly (e.g., disease screening: we want high recall to catch all sick patients).

- **FNR (Miss Rate):** $\frac{FN}{TP + FN} = 1 - \text{TPR}$

Rates computed from actual negatives:

- **FPR (False Positive Rate / Fall-out):** $\frac{FP}{TN + FP}$

“Of all actual negatives, what fraction did we falsely alarm?” Important when false alarms are costly (e.g., spam filter wrongly blocking important emails).

- **TNR (Specificity):** $\frac{TN}{TN + FP} = 1 - \text{FPR}$

Rates computed from predictions:

- **Precision (Positive Predictive Value):** $\frac{TP}{TP + FP}$

“Of all things we predicted positive, what fraction are actually positive?” Important when *acting on a positive prediction* is costly (e.g., if treatment is expensive or invasive).

Overall:

- **Accuracy:** $\frac{TP + TN}{N}$. **Misleading with class imbalance!** If 99% of emails are not spam, a classifier that always predicts “not spam” has 99% accuracy but 0% recall.
- **F1 Score:** $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$ (harmonic mean of precision and recall).

Key Idea 10.4 (Precision vs Recall Tradeoff). Precision and recall are in tension:

- **Lower threshold t** (predict positive more often): catches more positives ($\uparrow$ recall) but also more false alarms ($\downarrow$ precision).
- **Higher threshold t** (predict positive rarely): fewer false positives ($\uparrow$ precision) but misses more positives ($\downarrow$ recall).
- In the extreme: threshold $= -\infty$ gives recall $= 1$, precision $= n_+/n$; threshold $= +\infty$ gives recall $= 0$, precision $= 1$ (or undefined).

Which metric to prioritize depends on the application:

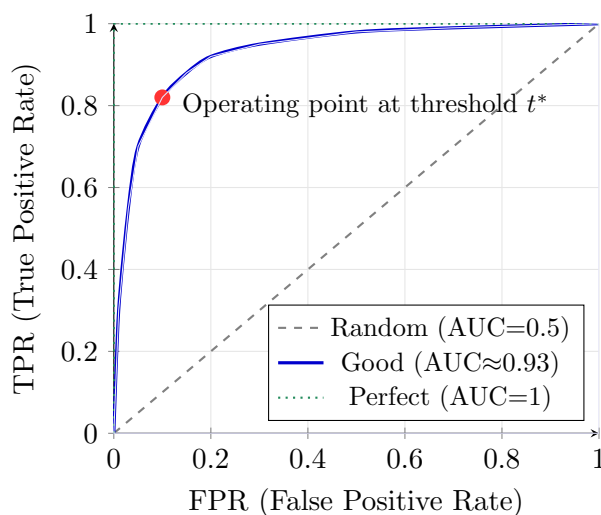
- **Cancer screening:** Prioritize **recall** (don't miss any cancer patients, even if some healthy people get extra tests).
- **Spam filter:** Prioritize **precision** (don't put important emails in spam, even if some spam gets through).
- **Balanced:** Use **F1 score** as a compromise.

10.6.2 ROC Curve

Definition 10.12 (ROC Curve and AUC). A **scoring classifier** outputs a real-valued score $s = \phi(\mathbf{x})$ (e.g., $s = \mathbf{w}^\top \mathbf{x}$ or $s = \sigma(\mathbf{w}^\top \mathbf{x})$). We predict $Y = 1$ iff $s > t$ for some threshold t . As t varies from $+\infty$ to $-\infty$:

- At $t = +\infty$: predict everything negative $\rightarrow$ TPR $= 0$, FPR $= 0$ (bottom-left corner).
- As t decreases: we start predicting more points positive $\rightarrow$ both TPR and FPR increase.
- At $t = -\infty$: predict everything positive $\rightarrow$ TPR $= 1$, FPR $= 1$ (top-right corner).

The **ROC curve** traces the path $(\text{FPR}(t), \text{TPR}(t))$ as t sweeps. A better classifier has its curve closer to the **top-left corner** (high TPR, low FPR).



Theorem 10.8 (AUC Probabilistic Interpretation).

$$\boxed{\text{AUC} = P[S^+ \geq S^-]}$$

where S^+ is the score of a random positive example and S^- is the score of a random negative example.

In words: If you pick one random positive and one random negative, AUC is the probability that the classifier scores the positive higher.

Key values:

- AUC = 1: positive scores always exceed negative scores (perfect separation).
- AUC = 0.5: scores of positives and negatives are identically distributed (no discriminative ability; equivalent to random guessing). ROC is the diagonal.
- AUC < 0.5: classifier is worse than random — flip its predictions!

Limitation: AUC measures *ranking quality* (can the classifier order positives above negatives?) but does NOT assess *calibration* (are predicted probabilities accurate?). A classifier with AUC = 0.9 might assign $p = 0.7$ to everything.

Example 10.4 (Building a ROC Curve Step-by-Step). Suppose we have 5 data points with scores and labels:

Point	A	B	C	D	E
Score	0.9	0.7	0.6	0.4	0.2
True label	+	+	−	+	−

Sweep threshold from high to low. 3 positives, 2 negatives.

1. $t > 0.9$: predict all $-$. TP=0, FP=0. Point: $(0/2, 0/3) = (0, 0)$.
2. $t = 0.9$: A becomes $+$. TP=1, FP=0. Point: $(0, 1/3)$.
3. $t = 0.7$: B becomes $+$. TP=2, FP=0. Point: $(0, 2/3)$.
4. $t = 0.6$: C becomes $+$ (false positive!). TP=2, FP=1. Point: $(1/2, 2/3)$.
5. $t = 0.4$: D becomes $+$. TP=3, FP=1. Point: $(1/2, 1)$.
6. $t = 0.2$: E becomes $+$. TP=3, FP=2. Point: $(1, 1)$.

The ROC curve connects these points. Each time a positive is added, the curve goes *up*; each time a negative is added, it goes *right*.

PR Curve: Plots Precision (y-axis) vs. Recall (x-axis). Better than ROC when classes are highly imbalanced, because FPR can be misleadingly low when negatives vastly outnumber positives.

Cost-sensitive classification: When errors have different costs (e.g., $C_{\text{FN}} \gg C_{\text{FP}}$ for cancer screening), choose the threshold on the ROC curve that minimizes expected cost: $\text{Cost} = C_{\text{FP}} \cdot \text{FP} + C_{\text{FN}} \cdot \text{FN}$.

Theorem 10.9 (Neyman-Pearson Lemma). Among all tests with false positive rate (size) at most α , the **likelihood ratio test**:

$$\phi_\eta(\mathbf{x}) = \mathbf{1} \left[\frac{p_1(\mathbf{x})}{p_0(\mathbf{x})} > \eta \right]$$

achieves the maximum true positive rate (power). This is the theoretically optimal binary classifier for a given FPR constraint. The corresponding ROC curve is the *upper envelope* of all possible classifiers.

11 Optimization

11.1 Batch Gradient Descent

Definition 11.1 (Gradient Descent Update).

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)})$$

where $\eta > 0$ is the **learning rate** (step size) and $\nabla E = \frac{1}{N} \sum_{n=1}^N \nabla L_n$ is the full gradient over all N data points.

Intuition: The gradient $\nabla E(\mathbf{w})$ points in the direction of steepest *increase*. By moving in the *opposite* direction ($-\nabla E$), we decrease the loss. The step size η controls how far we move.

Cost per update: $O(ND)$ — must compute gradient over all N data points.

Key Idea 11.1 (Gradient Descent for Different Loss Functions). The GD framework applies to *any* differentiable loss. Key examples:

- **OLS:** $E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, $\nabla E = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$. GD update: $\mathbf{w} \leftarrow \mathbf{w} - 2\eta\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$.
- **Ridge:** $E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2$, $\nabla E = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda\mathbf{w}$.
- **Logistic NLL:** $E(\mathbf{w}) = -\ell(\mathbf{w})$, $\nabla E = \mathbf{X}^\top(\mathbf{p} - \mathbf{y})$ (note sign flip since we minimize).

For OLS and Ridge, there's a closed-form solution so GD is unnecessary. But for logistic regression and other non-quadratic losses, GD (or its variants) is the standard approach.

11.2 Convergence Analysis (Detailed)

Theorem 11.1 (GD Convergence for Quadratic Loss (Full Derivation)). Consider the quadratic loss $E(\mathbf{w}) = \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top H(\mathbf{w} - \mathbf{w}^*)$ where H is PD with eigendecomposition $H = Q\Lambda Q^\top$ (Q orthogonal, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$).

Step 1: Gradient.

$$\nabla E(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*)$$

Step 2: GD update in original coordinates.

$$\mathbf{w}^{(\tau)} - \mathbf{w}^* = (I - \eta H)(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*)$$

Let $\mathbf{e}^{(\tau)} = \mathbf{w}^{(\tau)} - \mathbf{w}^*$ (error vector). Then $\mathbf{e}^{(\tau)} = (I - \eta H)^\tau \mathbf{e}^{(0)}$.

Step 3: Change to eigenbasis. Write $\mathbf{e}^{(0)} = \sum_i \alpha_i^{(0)} \mathbf{u}_i$ where $\mathbf{u}_i$ are eigenvectors of H . Then:

$$\mathbf{e}^{(\tau)} = \sum_i \alpha_i^{(0)} (1 - \eta \lambda_i)^\tau \mathbf{u}_i$$

Each eigencomponent evolves **independently**:

$$\alpha_i^{(\tau)} = (1 - \eta \lambda_i)^\tau \alpha_i^{(0)}$$

Step 4: Convergence condition. We need $|1 - \eta \lambda_i| < 1$ for all i so that each component decays to zero:

$$-1 < 1 - \eta \lambda_i < 1 \implies 0 < \eta \lambda_i < 2 \implies 0 < \eta < \frac{2}{\lambda_{\max}}$$

Step 5: Convergence rate. The slowest component has the largest $|1 - \eta \lambda_i|$. The **optimal learning rate** minimizes the worst-case convergence factor:

$$\eta^* = \frac{2}{\lambda_{\max} + \lambda_{\min}} \implies |1 - \eta^* \lambda_i|_{\max} = \frac{\kappa - 1}{\kappa + 1}$$

where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number. After τ steps:

$$\|\mathbf{e}^{(\tau)}\| \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^\tau \|\mathbf{e}^{(0)}\|$$

Definition 11.2 (Condition Number and Its Effects).

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Geometric interpretation: The condition number measures how *elongated* the loss contours are:

- $\kappa = 1$: contours are perfect circles. GD goes straight to the minimum.
- $\kappa \gg 1$: contours are narrow, elongated ellipses. GD *zigzags* across the narrow direction, making slow progress along the long direction.

Convergence speed: To reduce error by factor ϵ , need $\tau \approx \frac{\kappa}{2} \log(1/\epsilon)$ steps.

For OLS: $H = 2X^\top X$, so $\kappa = \sigma_{\max}^2 / \sigma_{\min}^2$ where σ_i are singular values of X .

Ridge improves conditioning: $\kappa_{\text{ridge}} = \frac{\sigma_{\max}^2 + \lambda}{\sigma_{\min}^2 + \lambda} < \kappa_{\text{OLS}}$. Ridge makes the contours more circular.

Example 11.1 (Why Ill-Conditioning Causes Zigzagging). Consider $E(w_1, w_2) = \frac{1}{2}(100w_1^2 + w_2^2)$. Here $\lambda_1 = 100$, $\lambda_2 = 1$, $\kappa = 100$.

The dilemma: no learning rate works well for both directions.

Try $\eta = 0.01$ (small enough for stability in the steep direction):

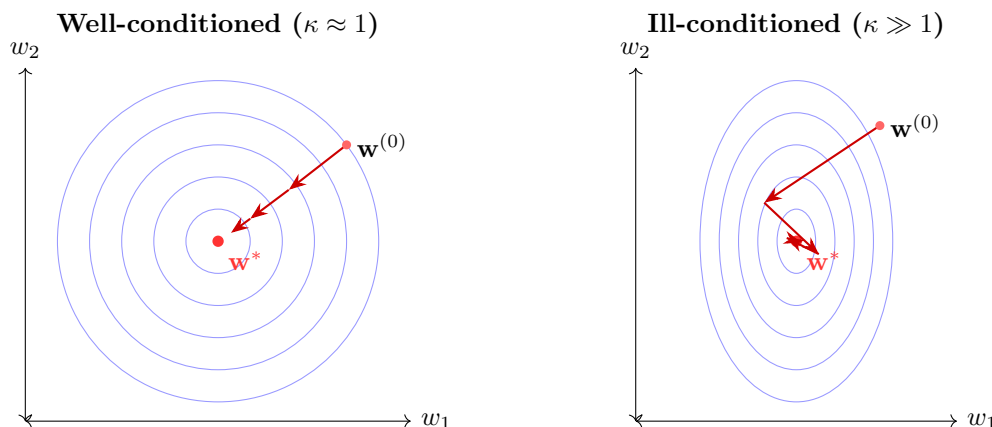
- Along w_1 (steep, $\lambda_1 = 100$): $|1 - 0.01 \cdot 100| = |1 - 1| = 0$. **Perfect!** Converges in one step.
- Along w_2 (flat, $\lambda_2 = 1$): $|1 - 0.01 \cdot 1| = 0.99$. **Painfully slow.** After 100 steps, error is still $0.99^{100} \approx 0.37$ of the original.

Try $\eta = 0.019$ (larger, to speed up w_2):

- Along w_1 : $|1 - 0.019 \cdot 100| = |1 - 1.9| = 0.9$. Converges, but the factor is *negative* ($1 - 1.9 = -0.9$), so the error **flips sign every step** — w_1 oscillates/zigzags.
- Along w_2 : $|1 - 0.019| = 0.981$. Still slow!

The optimal $\eta^* = 2/(\lambda_{\max} + \lambda_{\min}) = 2/101 \approx 0.0198$ equalizes the convergence factors: both become $(\kappa - 1)/(\kappa + 1) = 99/101 \approx 0.98$. But the steep direction *zigzags* (negative factor) while the flat direction *crawls* (positive factor close to 1).

Key insight: With a single learning rate, you can't make *both* directions converge fast when $\kappa \gg 1$. This is exactly what momentum (damps zigzags) and adaptive methods (per-parameter η) fix.



Key Idea 11.2 (Learning Rate: Too Large vs Too Small). • **η too small:** Convergence is guaranteed but extremely slow. Each step makes tiny progress. Wastes computation.

- **η just right:** Fast convergence. For quadratic, optimal $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$.
- **η too large ($\eta > 2/\lambda_{\max}$):** GD *diverges*. The updates overshoot the minimum and the loss grows without bound.
- **η slightly too large:** GD oscillates around the minimum without converging.

Example 11.2 (GD Convergence for Linear Regression). For OLS with loss $E(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$:

- Gradient: $\nabla E = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$
- Hessian: $H = 2\mathbf{X}^\top\mathbf{X}$ (constant — the loss is exactly quadratic!)
- The eigenvalues of H are $2\sigma_i^2$ where σ_i are the singular values of $\mathbf{X}$.
- Convergence condition: $\eta < 2/(2\sigma_{\max}^2) = 1/\sigma_{\max}^2$
- GD converges to the exact OLS solution $\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$.

For linear regression, GD finds the same answer as the normal equations — the loss is quadratic, so there's a single global minimum and GD converges to it.

11.3 Stochastic Gradient Descent (SGD)

Definition 11.3 (SGD). Instead of computing the gradient over all N points, use a *single* randomly chosen point:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla L_n(\mathbf{w}^{(\tau-1)})$$

where n is randomly sampled from $\{1, \dots, N\}$. **Cost per step:** $O(D)$ instead of $O(ND)$.

Theorem 11.2 (SGD is Unbiased). The single-point gradient ∇L_n is an *unbiased estimate* of the full gradient:

$$\mathbb{E}_n[\nabla L_n(\mathbf{w})] = \frac{1}{N} \sum_{n=1}^N \nabla L_n(\mathbf{w}) = \nabla E(\mathbf{w})$$

Proof: If n is uniformly random over $\{1, \dots, N\}$, then $\mathbb{E}[\nabla L_n] = \frac{1}{N} \sum_n \nabla L_n = \nabla E$. So SGD follows the correct direction *on average*, but each step is noisy. The noise has variance:

$$\text{Var}[\nabla L_n] = \frac{1}{N} \sum_{n=1}^N \|\nabla L_n - \nabla E\|^2$$

This variance does NOT vanish as we approach the optimum (unlike the full gradient, which goes to zero).

Key Idea 11.3 (SGD Convergence Properties). • **Fixed η :** SGD does not converge to the exact minimum. Instead, it “bounces around” in a region of size $\sim \eta \cdot \text{noise}$ around the optimum.

- **Decreasing η :** With $\eta^{(\tau)} \rightarrow 0$ slowly enough (e.g., $\eta^{(\tau)} = \eta_0/\tau$), SGD converges to the true minimum. The conditions are: $\sum_{\tau} \eta^{(\tau)} = \infty$ (reach anywhere) and $\sum_{\tau} (\eta^{(\tau)})^2 < \infty$ (noise dies out).
- **Stochastic noise is sometimes beneficial:** The noise helps escape shallow local minima and saddle points that trap batch GD.

Definition 11.4 (Mini-Batch SGD). Use a random subset (batch) of B points:

$$\nabla E_B = \frac{1}{B} \sum_{b=1}^B \nabla L_b, \quad \text{cost: } O(BD)$$

Epoch: One complete pass over all N data points (N/B gradient steps per epoch).

Variance reduction: $\text{Var}[\nabla E_B] = \frac{1}{B} \text{Var}[\nabla L_n]$ — variance decreases linearly with batch size. Tradeoffs:

- **Larger batch ($B \rightarrow N$):** Better gradient estimate, more hardware parallelism, smoother convergence. But diminishing returns: doubling B halves variance but doubles cost.
- **Smaller batch ($B = 1$):** Cheapest per step, most stochastic (helps escape local minima). But gradient estimate is very noisy.
- **Sweet spot:** $B = 32$ to 512 typical in practice. Balances noise level with GPU utilization.

11.4 Momentum (Detailed Derivation)

Definition 11.5 (Momentum). Introduce a **velocity** variable $\mathbf{v}$ that accumulates past gradients:

$$\mathbf{v}^{(\tau)} = \mu \cdot \mathbf{v}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)})$$

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \mathbf{v}^{(\tau)}$$

where $\mu \in [0, 1)$ is the momentum coefficient (typically $\mu = 0.9$). Initialize $\mathbf{v}^{(0)} = \mathbf{0}$.

Theorem 11.3 (Momentum Derivation: Effective Learning Rate). **Expanding the recursion:** The velocity at time τ is:

$$\mathbf{v}^{(\tau)} = -\eta \sum_{k=0}^{\tau-1} \mu^k \nabla E(\mathbf{w}^{(\tau-1-k)})$$

This is an *exponentially weighted sum* of past gradients, with recent gradients weighted more.

Case 1: Consistent gradient direction. If $\nabla E = g$ (constant) in some direction, then:

$$v_{\infty} = -\eta g(1 + \mu + \mu^2 + \dots) = \frac{-\eta g}{1 - \mu}$$

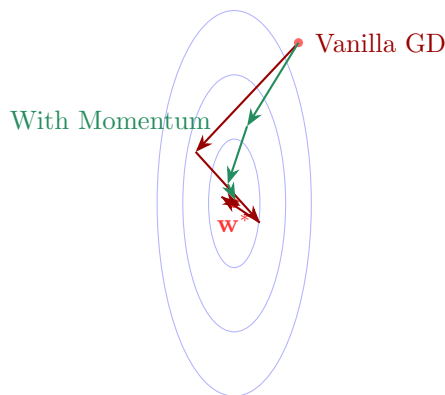
The effective step size is $\frac{\eta}{1-\mu}$. With $\mu = 0.9$: $10\times$ speedup!

Case 2: Oscillating gradient. If the gradient alternates sign $(g, -g, g, -g, \dots)$:

$$v_{\infty} = -\eta g(1 - \mu + \mu^2 - \mu^3 + \dots) = \frac{-\eta g}{1 + \mu}$$

The effective step size is $\frac{\eta}{1+\mu}$. With $\mu = 0.9$: $\sim 0.53\times$ — **oscillations are damped**.

Net effect: Momentum accelerates motion along the “valley” while dampening oscillations across the valley walls. This is exactly what’s needed for ill-conditioned problems.



11.5 Adaptive Learning Rate Methods

Definition 11.6 (AdaGrad (Detailed)). **Idea:** Give each parameter its own learning rate, scaled inversely by the historical gradient magnitude.

Accumulate squared gradients:

$$r_i^{(\tau)} = r_i^{(\tau-1)} + (g_i^{(\tau)})^2 = \sum_{k=1}^{\tau} (g_i^{(k)})^2$$

Update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \epsilon}} \cdot g_i^{(\tau)}$$

Effect: Parameters with large historical gradients (steep directions) get smaller learning rates. Parameters with small historical gradients (flat directions) get larger learning rates.

Problem: r_i is a *monotonically increasing sum* — it only grows. After many steps, $r_i \rightarrow \infty$ and the effective learning rate $\eta/\sqrt{r_i} \rightarrow 0$ for ALL parameters, even if the gradient is still large. This causes training to stall. Good for convex problems (where you want to slow down near the optimum), problematic for non-convex.

Definition 11.7 (RMSProp (Detailed Derivation)). **Fix AdaGrad's problem** by replacing the cumulative sum with an **exponentially weighted moving average** (EWMA) of squared gradients:

$$r_i^{(\tau)} = \beta \cdot r_i^{(\tau-1)} + (1 - \beta) \cdot (g_i^{(\tau)})^2$$

Typical: $\beta = 0.9$ (window of ~ 10 steps), $\beta = 0.99$ (window of ~ 100 steps).

Why this works: Expanding the recursion:

$$r_i^{(\tau)} = (1 - \beta) \sum_{k=0}^{\tau-1} \beta^k (g_i^{(\tau-k)})^2$$

Old gradients are exponentially forgotten. The *effective window* has length $\sim 1/(1 - \beta)$. The learning rate adapts to the *recent* gradient magnitude, not the entire history.

Update: Same as AdaGrad: $w_i = \frac{\eta}{\sqrt{r_i + \epsilon}} g_i$

Definition 11.8 (Adam (Full Derivation)). Adam = **A**daptive **M**oment estimation. Combines the best of Momentum and RMSProp.

Step 1: First moment estimate (momentum on gradients):

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1 - \beta_1) g_i^{(\tau)} \quad (\text{default: } \beta_1 = 0.9)$$

This is an EWMA of the gradient. Estimates $\mathbb{E}[g_i]$.

Step 2: Second moment estimate (RMSProp):

$$r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1 - \beta_2) (g_i^{(\tau)})^2 \quad (\text{default: } \beta_2 = 0.999)$$

This is an EWMA of the squared gradient. Estimates $\mathbb{E}[g_i^2]$.

Step 3: Bias correction. Both s and r are initialized to 0, creating a bias toward 0 in early steps. The expected value of s after τ steps is:

$$\mathbb{E}[s_i^{(\tau)}] = (1 - \beta_1^\tau) \mathbb{E}[g_i]$$

Proof: Expanding $s_i^{(\tau)} = (1 - \beta_1) \sum_{k=1}^{\tau} \beta_1^{\tau-k} g_i^{(k)}$. Taking expectation (assume stationary g):

$$\mathbb{E}[s_i^{(\tau)}] = (1 - \beta_1) \mathbb{E}[g_i] \sum_{k=0}^{\tau-1} \beta_1^k = (1 - \beta_1^\tau) \mathbb{E}[g_i].$$

So we divide by $(1 - \beta_1^\tau)$ to get an unbiased estimate:

$$\hat{s}_i = \frac{s_i^{(\tau)}}{1 - \beta_1^\tau}, \quad \hat{r}_i = \frac{r_i^{(\tau)}}{1 - \beta_2^\tau}$$

Note: β_2^τ decays slowly ($\beta_2 = 0.999$), so bias correction matters for hundreds of steps.

Step 4: Update:

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \cdot \frac{\hat{s}_i}{\sqrt{\hat{r}_i} + \delta}$$

Default hyperparameters: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\delta = 10^{-8}$.

Key Idea 11.4 (Why Adam Works So Well). Adam adapts the learning rate **per-parameter**:

- $\hat{s}_i \approx \mathbb{E}[g_i]$: the momentum term smooths out noise and accumulates direction.
- $\sqrt{\hat{r}_i} \approx \text{RMS}(g_i)$: normalizes by the typical gradient magnitude.
- The update $\hat{s}_i / \sqrt{\hat{r}_i}$ is roughly the *signal-to-noise ratio* of the gradient.
- Parameters with consistent, large gradients get effective step $\approx \eta$.
- Parameters with noisy gradients get reduced effective step.

Method	Update rule	Per-param LR?	Momentum?
GD	$\mathbf{w} - \eta \nabla E$	No	No
SGD	$\mathbf{w} - \eta \nabla L_n$	No	No
Momentum	$\mathbf{w} + \mu \mathbf{v} - \eta \nabla E$	No	Yes
AdaGrad	$w_i - \frac{\eta}{\sqrt{r_i}} g_i$	Yes	No
RMSPprop	$w_i - \frac{\eta}{\sqrt{r_i^{\text{ema}}}} g_i$	Yes	No
Adam	$w_i - \eta \frac{\hat{s}_i}{\sqrt{\hat{r}_i}}$	Yes	Yes

11.6 Learning Rate Schedules

Instead of a fixed learning rate, decay it over time:

- **Cosine:** $\eta^{(\tau)} = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi\tau/T))$ (common for LLMs)
- **Power-law:** $\eta^{(\tau)} = \eta_0(1 + \tau/s)^{-c}$
- **Exponential:** $\eta^{(\tau)} = \eta_0 \cdot c^{\lfloor \tau/s \rfloor}$
- **Warm-up:** Start with small η and linearly increase to $\eta_{\max}$ over the first few epochs, then decay. Prevents early divergence.

12 Loss Functions for Classification

Understanding which loss function to use for classification is critical. Different losses have different properties, gradients, and effects on training.

12.1 The Three Main Classification Losses

Definition 12.1 (Squared Loss (for Classification)). Using $y \in \{0, 1\}$ and prediction $p = \sigma(\mathbf{w}^\top \mathbf{x})$:

$$L_{\text{sq}}(y, p) = (y - p)^2$$

Gradient w.r.t. p : $\frac{\partial L}{\partial p} = -2(y - p)$

Issues for classification:

- When the model is very wrong (e.g., $y = 1$ but $p \approx 0$): the gradient is ≈ -2 , which is *bounded*. The model doesn't get a strong "push" to fix very wrong predictions.
- When $y = 1$ and $p > 1$ (can't happen with sigmoid, but with raw scores): the loss *increases*, penalizing the model for being too confident even when correct!
- Squared loss treats all errors equally—but in classification, being wrong with high confidence should be penalized much more than being slightly uncertain.

Definition 12.2 (Binary Cross-Entropy Loss (= Logistic Loss = Log Loss)).

$$L_{\text{BCE}}(y, p) = -[y \log p + (1 - y) \log(1 - p)]$$

where $y \in \{0, 1\}$ and $p = \sigma(\mathbf{w}^\top \mathbf{x}) \in (0, 1)$.

This is the same as the negative log-likelihood of logistic regression!

Substituting $p = \sigma(z)$ where $z = \mathbf{w}^\top \mathbf{x}$:

$$L_{\text{BCE}}(y, z) = -y \log \sigma(z) - (1 - y) \log \sigma(-z) = \log(1 + e^{-z}) \cdot y + \log(1 + e^z) \cdot (1 - y)$$

Using $\tilde{y} = 2y - 1 \in \{-1, +1\}$:

$$L_{\text{logistic}}(\tilde{y}, z) = \log(1 + e^{-\tilde{y}z})$$

Gradient w.r.t. z : $\frac{\partial L}{\partial z} = p - y = \sigma(z) - y$. This is exactly the logistic regression gradient!

Key properties:

- As $p \rightarrow 0$ when $y = 1$: $L \rightarrow +\infty$. Very wrong predictions are *severely* penalized.
- As $p \rightarrow 1$ when $y = 1$: $L \rightarrow 0$. Correct predictions have near-zero loss.
- The gradient is *large* when the model is very wrong, giving a strong learning signal.
- The loss is **convex** in $z = \mathbf{w}^\top \mathbf{x}$ (even though it's applied after the sigmoid).

Key Idea 12.1 (Binary Cross-Entropy IS Logistic Loss). These are **the same loss** written in different ways:

- **Binary cross-entropy:** $-[y \log p + (1 - y) \log(1 - p)]$ with $y \in \{0, 1\}$, $p \in (0, 1)$.
- **Logistic loss:** $\log(1 + e^{-\tilde{y}z})$ with $\tilde{y} \in \{-1, +1\}$, $z \in \mathbb{R}$.
- **NLL of logistic regression:** $-\ell_i(\mathbf{w})$ for a single data point.

All three are the same function. The naming depends on the field (information theory vs. ML vs. statistics).

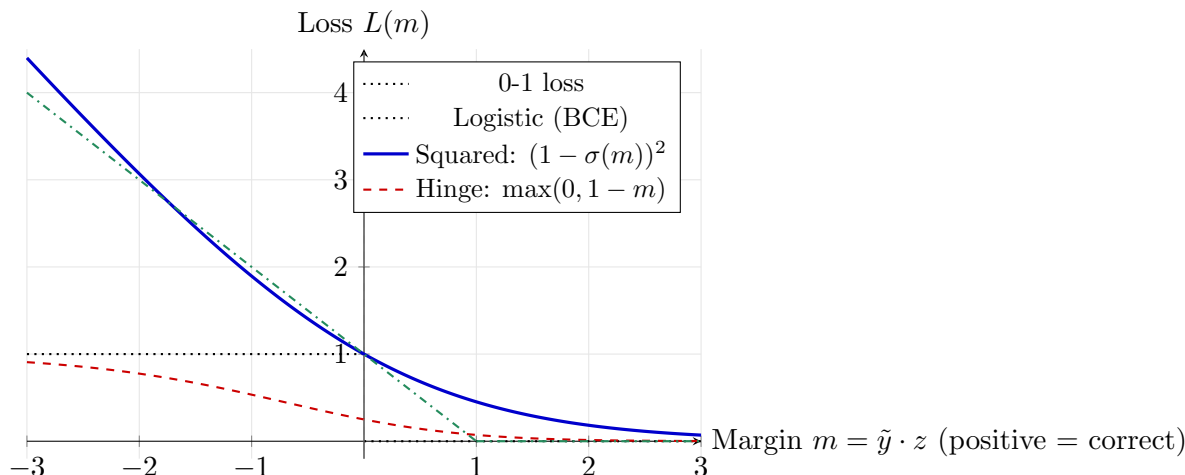
Definition 12.3 (Hinge Loss (for reference)). Used in SVMs (not the focus of this course, but useful for comparison):

$$L_{\text{hinge}}(\tilde{y}, z) = \max(0, 1 - \tilde{y}z)$$

The gradient is 0 when $\tilde{y}z > 1$ (correctly classified with margin) and $-\tilde{y}$ otherwise. Not differentiable at $\tilde{y}z = 1$.

12.2 Comparing Loss Functions Graphically

The plot below shows each loss as a function of the “margin” $m = \tilde{y}z$ (positive = correct, negative = wrong):



Key Idea 12.2 (Loss Function Comparison). • **0-1 loss** (ideal but non-differentiable): 1 if wrong, 0 if correct. Can't optimize with gradients.

- **Logistic/BCE loss**: Smooth, convex upper bound on 0-1 loss. Strong gradient signal for wrong predictions. Goes to ∞ as confidence in the wrong answer grows. This is why it's the standard for classification.
- **Squared loss**: Bounded gradient even for very wrong predictions. Also penalizes “too confident” correct predictions (the loss curve flattens and then rises again for very large correct margins). Poor for classification.
- **Hinge loss**: Linear penalty for wrong predictions (even stronger gradient than logistic). Zero loss for correctly classified points with sufficient margin. Used in SVMs.

Bottom line: Use **binary cross-entropy (logistic loss)** for classification. It's the natural loss from MLE with Bernoulli likelihood, it's convex, and it provides strong gradients for wrong predictions.

12.3 Multi-Class Cross-Entropy Loss

For K classes with softmax predictions $p_k = \text{softmax}(\mathbf{z})_k$ and one-hot label $\mathbf{y}$:

$$L_{\text{CE}} = - \sum_{k=1}^K y_k \log p_k = - \log p_{y^*}$$

where $y^* = \text{argmax}_k y_k$ is the true class. This is equivalent to negative log-likelihood of the multinomial.

Connection to KL divergence: Since $H(\mathbf{y}) = 0$ for one-hot labels:

$$L_{\text{CE}} = H(\mathbf{y}, \mathbf{p}) = H(\mathbf{y}) + D_{\text{KL}}(\mathbf{y} \parallel \mathbf{p}) = D_{\text{KL}}(\mathbf{y} \parallel \mathbf{p})$$

So minimizing cross-entropy = minimizing KL divergence from true to predicted distribution.

13 Information Theory

13.1 Entropy

Definition 13.1 (Shannon Entropy). The entropy of a discrete random variable X with distribution p :

$$H(X) = - \sum_x p(x) \log p(x)$$

Convention: $0 \log 0 := 0$.

Interpretation: The minimum average number of bits needed to encode samples from X . Higher entropy = more uncertainty/randomness.

- **Maximum entropy:** Uniform distribution: $H = \log |\mathcal{X}|$.
- **Minimum entropy:** Deterministic distribution (one outcome has probability 1): $H = 0$.

13.2 KL Divergence

Definition 13.2 (Kullback-Leibler Divergence).

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0$$

Properties:

- $D_{\text{KL}}(p||q) = 0$ if and only if $p = q$.
- $D_{\text{KL}}(p||q) \geq 0$ (**Gibbs' inequality**).
- **NOT symmetric:** $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$ in general.
- If $p(x) > 0$ but $q(x) = 0$: $D_{\text{KL}} = +\infty$.

Interpretation: Extra bits wasted when using a code designed for distribution q when the true distribution is p .

Example 13.1 (Asymmetry of KL Divergence). Let $p = \text{fair coin } ([1/2, 1/2])$ and $q = \text{deterministic } ([1, 0])$:

- $D(p||q) = \frac{1}{2} \log \frac{1/2}{1} + \frac{1}{2} \log \frac{1/2}{0} = +\infty$ (p puts mass where q has zero!)
- $D(q||p) = 1 \cdot \log \frac{1}{1/2} + 0 \cdot \log \frac{0}{1/2} = \log 2 = 1$ bit

13.3 Cross-Entropy

Definition 13.3 (Cross-Entropy).

$$H(p, q) = - \sum_x p(x) \log q(x) = H(p) + D_{\text{KL}}(p \| q)$$

Interpretation: Total average bits to encode data from distribution p using code designed for q .

- $H(p, q) \geq H(p)$, with equality iff $p = q$.
- NOT symmetric: $H(p, q) \neq H(q, p)$.

Binary cross-entropy:

$$H(p, q) = -[p \log q + (1 - p) \log(1 - q)]$$

Key Idea 13.1 (Cross-Entropy Loss = Negative Log-Likelihood). For logistic regression with true labels y_i and predicted probabilities $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$:

$$-\text{LL}(\mathbf{w}) = - \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)] = \sum_i H([y_i, 1 - y_i], [p_i, 1 - p_i])$$

Minimizing the cross-entropy loss is equivalent to maximizing the log-likelihood. Since $H(p) = 0$ for one-hot p (labels are deterministic), minimizing cross-entropy is the same as minimizing KL divergence between true and predicted distributions.

14 Key Tricks, Patterns, and Problem-Solving Techniques

This section collects important techniques from discussions and homeworks that are commonly tested.

14.1 Sigmoid Identities

These come up constantly in logistic regression derivations:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{30}$$

$$1 - \sigma(z) = \sigma(-z) \tag{31}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{32}$$

$$\frac{\sigma(z)}{1 - \sigma(z)} = e^z \implies \log \frac{\sigma(z)}{1 - \sigma(z)} = z \tag{33}$$

$$\text{For } f(\mathbf{x}) = \sigma(\mathbf{a}^\top \mathbf{x}) : \quad \frac{\partial f}{\partial x_i} = a_i \sigma(\mathbf{a}^\top \mathbf{x})(1 - \sigma(\mathbf{a}^\top \mathbf{x})) \tag{34}$$

14.2 Gradient Computation Patterns

- $\nabla_{\mathbf{w}}(\mathbf{a}^\top \mathbf{w}) = \mathbf{a}; \quad \nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{w}) = 2\mathbf{w}$
- $\nabla_{\mathbf{w}}(\mathbf{w}^\top A \mathbf{w}) = (A + A^\top)\mathbf{w}; \quad \text{if } A \text{ symmetric: } 2A\mathbf{w}$

- $\nabla_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|^2 = 2X^\top(X\mathbf{w} - \mathbf{y})$ (the most important one!)
- Hessian of $\|X\mathbf{w} - \mathbf{y}\|^2$ is $2X^\top X$
- For WLS: $\nabla_{\mathbf{w}}(\mathbf{y} - X\mathbf{w})^\top A(\mathbf{y} - X\mathbf{w}) = -2X^\top A(\mathbf{y} - X\mathbf{w})$

14.3 Normal Equations Pattern

Setting gradients to zero in least-squares problems always yields the pattern:

$$(X^\top X + \text{regularizer})^{-1} X^\top \mathbf{y}$$

- OLS: $(X^\top X)^{-1} X^\top \mathbf{y}$
- Ridge: $(X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$
- WLS: $(X^\top A X)^{-1} X^\top A \mathbf{y}$

14.4 Residual Orthogonality

From the normal equations $X^\top X \mathbf{w}^* = X^\top \mathbf{y}$:

$$X^\top (\mathbf{y} - X\mathbf{w}^*) = \mathbf{0} \implies X^\top \mathbf{r} = \mathbf{0}$$

The residual vector is orthogonal to every column of X . If X has a bias column (column of 1s), then $\mathbf{1}^\top \mathbf{r} = 0$, so the residuals sum to zero.

14.5 Chain Rule for Deep/Recursive Systems

For a recursive system $z_k = w_{k-1}z_{k-1} + b_{k-1}$:

$$\frac{dz_n}{dz_1} = \prod_{j=1}^{n-1} w_j, \quad \frac{dz_n}{db_1} = \prod_{j=2}^{n-1} w_j$$

This is the mathematical basis for **vanishing/exploding gradients** in deep networks:

- All $|w_j| < 1$: gradient vanishes exponentially.
- All $|w_j| > 1$: gradient explodes exponentially.

14.6 PSD/PD Proof Technique

To show A is PSD, use the spectral decomposition $A = Q\Lambda Q^\top$:

$$\mathbf{x}^\top A \mathbf{x} = \mathbf{x}^\top Q \Lambda Q^\top \mathbf{x} = \sum_i \lambda_i (q_i^\top \mathbf{x})^2$$

Since $(q_i^\top \mathbf{x})^2 \geq 0$, this is ≥ 0 iff all $\lambda_i \geq 0$.

To show $A^\top A$ is PSD: $\mathbf{x}^\top A^\top A \mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$.

14.7 Overparameterized Systems and Min-Norm Solutions

When $d > n$ (more parameters than data), there are infinitely many solutions to $X\mathbf{w} = \mathbf{y}$. Gradient descent from $\mathbf{w}_0 = \mathbf{0}$ converges to the **minimum-norm solution**:

$$\mathbf{w}^* = X^\top (XX^\top)^{-1} \mathbf{y}$$

This is because each GD update adds a vector in the row space of X ($\text{range}(X^\top)$), so $\mathbf{w}$ always stays in the row space. The minimum-norm solution is the unique solution in the row space.

14.8 Likelihood Ratio and Capture-Recapture MLE

For discrete parameters (like population size N), use the **likelihood ratio method**:

$$R(N | x) = \frac{L(N; x)}{L(N-1; x)}$$

The MLE is the value of N where R crosses 1. For capture-recapture: $\hat{N} = nk/x$ where n = first sample, k = second sample, x = recaptured.

14.9 Tail Probability Formula for Expectation

For a non-negative random variable X :

$$\mathbb{E}[X] = \int_0^\infty P(X > t) dt$$

This is useful when the CDF is easier to work with than the PDF directly.

14.10 Robust Methods

- **Huber loss:** Quadratic for small residuals, linear for large ones. More robust than squared loss.
- **Cauchy contamination:** Even a small fraction of Cauchy-distributed data makes $\mathbb{E}[(x - c)^2] = \infty$ for all c . Need robust methods (hard thresholding, Huber loss, or trimmed means).
- **L1 loss:** Minimizer is median (robust to outliers). L2 loss minimizer is mean (sensitive to outliers).

14.11 p -Norms

$$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p \right)^{1/p}$$

- $\|\mathbf{x}\|_0 = \#\{i : x_i \neq 0\}$ (counts non-zeros; not a true norm)
- $\|\mathbf{x}\|_1 = \sum |x_i|$ (Manhattan distance; diamond-shaped unit ball)
- $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$ (Euclidean distance; circular unit ball)
- $\|\mathbf{x}\|_\infty = \max |x_i|$ (max norm; square-shaped unit ball)
- $p < 1$: non-convex unit ball

14.12 Model Relationships Summary

GDA $\xrightarrow{\text{equal } \Sigma}$ LDA $\rightarrow$ sigmoid $\rightarrow$ Logistic Regression

GDA $\xrightarrow{\text{unequal } \Sigma}$ QDA (quadratic boundary)

GDA $\xrightarrow{\text{diagonal } \Sigma}$ Naive Bayes

LDA $\subseteq$ Logistic Regression (LDA is a special case)

K-Means = zero-variance limit of GMM

14.13 Regularization Summary

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \ \mathbf{w}\ _2^2$	$\lambda \ \mathbf{w}\ _1$
Effect	Shrinks all weights	Sets some weights to 0
Sparsity	No	Yes
Prior	Gaussian	Laplace
Closed form	Yes	No
Correlated features	Handles well	Picks one, ignores rest
$\lambda = ?$	σ^2 / τ^2	$2\sigma^2 / b$
Constraint shape	Circle (smooth)	Diamond (corners)